Article

Kernel Borderlands: An eBPF-Based Behavioral Detection and Multi-Agent Containment Architecture for Linux Server Workloads

Pardhu Sri Rushi Varma Konduru *, Tejaswini Kandukoori, Karthik Kanne and Rupa Yeshvitha Karedla

Department of Cybersecurity, Malla Reddy University, Hyderabad 500100, Telangana, India;
2311CS040089@mallareddyuniversity.ac.in (P.S.R.V.K.); 2311CS040077@mallareddyuniversity.ac.in (T.K.);
2311CS040080@mallareddyuniversity.ac.in (K.K.); 2311CS040082@mallareddyuniversity.ac.in (R.Y.K.)
* Correspondence: 2311CS040089@mallareddyuniversity.ac.in

Abstract

Linux servers are usually defended by tools that observe kernel activity but leave the response to a human, or by static policies that cannot recognise a hijacked legitimate binary. We present Kernel Borderlands (KB), a kernel-level runtime defence that closes the loop from observation to enforcement. An eBPF sensor scores process behaviour and enforces five graded containment levels, the first four reversible, through BPF-LSM hooks, behind authorisation gates that prevent it from acting against PID 1, critical daemons or its own components. A Go control plane keeps a SHA-256 hash-chained audit log and detects beaconing; a Ray-based agent swarm implements a containment policy trained by Proximal Policy Optimisation and quorum confirmation of actions; and a Rust watchdog verifies BPF bytecode and gates the boot sequence. On 908 held-out samples derived from ADFA-LD, the containment policy reaches 97.1–100% accuracy on every attack class with no under-containment, and the Jury policy has no false negatives. In a live capture, the control plane issued a throttle within about 1 ms of each of six escalations to BORDERLANDS. With all hooks loaded, KB adds 38 ns per system call and 26 μs per process creation, with no measurable overhead on a parallel compile or a CPU-bound workload.

Keywords: eBPF; BPF-LSM; runtime containment; intrusion detection; reinforcement learning; multi-agent systems

1. Introduction

When a Linux server is compromised, the attacker's actions eventually become kernel operations: a process is executed, a credential changes, a sensitive file is opened, a socket connects to a remote host, or a memory region becomes writable and executable. The extended Berkeley Packet Filter (eBPF) lets verified programs observe these operations at Ring 0 with low overhead [1], and since Linux 5.7 BPF-LSM programs can also *deny* them [2].

Observation, however, is not defence. A runtime defence must also *decide* how strongly to respond, *enforce* that decision where the attacker cannot simply ignore it, *prove* afterwards that its record of what it did has not been altered, and *protect itself* from being blinded or turned against the host. Existing tools usually provide one or two of these properties: eBPF monitors alert but rarely respond, enforcement tools respond with a fixed kill-or-deny rule, and almost none keep a tamper-evident record of their own actions.

Transitions, not events.

A single `execve()` is not an attack, and neither is a single anonymous `mmap()` or a single change of user ID. The same `execve()` after a privilege change, inside an unexpected process lineage and after

a burst of writable-and-executable mappings very likely is. KB therefore treats the *transition* between behavioural states over time as its unit of analysis: each event updates one of six weighted behavioural dimensions of its process, the composite score is smoothed, and rule chains move the process through the states SAFE, OBSERVED, SUSPICIOUS, BORDERLANDS and COMPROMISED.

Problem statement.

From this view we derive five problems that existing tools do not solve together.

P1 *Observability is not response.* Seeing an attack is of little use unless the system can choose and apply a proportionate response inside the kernel, quickly and, where possible, reversibly.

P2 *Telemetry must never delay enforcement.* A burst of events, for example from a build job that spawns thousands of short-lived processes, must not stall the path that carries containment orders. We hit exactly this failure during development: on one shared socket a telemetry burst filled the send buffer, the sensor misread `EAGAIN` as a dead peer, and containment delivery went down with it.

P3 *The defence must not be turnable against itself.* The containment pipeline must refuse to act on PID 1, kernel threads, critical daemons, organisation-critical workloads or its own watchdog, even if an upstream decision asks it to.

P4 *Decisions must be provable.* Every enforcement action, policy reload and agent decision must be recorded so that later modification or deletion is detectable.

P5 *The decision-makers need supervision.* Once agents may recommend containment, a stuck or misbehaving agent must be detected and restarted, and the decision layer should not start until the layers beneath it have been verified.

Kernel Borderlands (KB) implements the full observe–score–decide–enforce–audit loop for Linux server workloads. Its name reflects the central idea: each process lives in a behavioural *zone* (SAFE, SUSPICIOUS or BORDERLANDS), and the defence acts at the border between normal and hostile behaviour rather than on individual events. The contributions of this paper are:

1. An architecture of five cooperating subsystems connected only through root-owned Unix sockets, with telemetry and control on separate channels (§4).
2. Five graded containment levels enforced by BPF-LSM, guarded by two ordered authorisation gates (CPM, CWP) that no decision layer can bypass (§6).
3. A unified formulation of every learned and statistical decision procedure: behaviour scoring, the reinforcement-learned Containment and Jury policies, quorum consensus, agent oversight, beaconing detection and the audit chain (§5).
4. An evaluation combining offline policy results, a live attack capture, a run-time overhead benchmark, the test suites, and a source-level security audit, together with the defects found by testing on real hosts (§14–§16).
5. An open-source implementation packaged for Ubuntu and Arch Linux, with its kernel requirements, portability model and a test case study (§11, §14.4).

Section 2 reviews related work and Section 3 states the threat model. Sections 4–9 describe the architecture, the mathematical formulation and each subsystem; Sections 10–13 cover implementation, deployment, the end-to-end path and the operator interfaces. Section 14 evaluates the system, and Sections 15–18 discuss the results, the audit, the limitations and future work.

2. Background and Related Work

eBPF and BPF-LSM.

The BSD Packet Filter [3] introduced an in-kernel virtual machine for packet filtering. eBPF generalised it with maps, helper functions and a verifier that proves termination and memory safety before a program is loaded [1]; programs attach to tracepoints, kprobes, XDP [4] and, through KRSI, LSM hooks [2]. BTF and CO-RE let one compiled object run unchanged across kernel versions [5].

Table 1. Capabilities of Linux runtime-defence approaches

Capability	MAC	auditd	Falco	Tracee	Tetragon	**KB**
Kernel vantage point	✓	part.	✓	✓	✓	✓
Behavioural sequences	–	–	rules	✓	rules	✓
In-kernel enforcement	✓	–	–	–	✓	✓
Graded containment levels	–	–	–	–	–	5
Learned decision policy	–	–	–	–	–	PPO
Quorum before acting	–	–	–	–	–	✓
Self-protection gates	part.	–	–	–	–	✓
Tamper-evident audit	–	part.	–	–	–	✓
Boot-gating watchdog	–	–	–	–	–	✓

Access control and intrusion detection.

SELinux [6] and AppArmor enforce static policy through the LSM framework [7]; an exploited server that stays within its permitted operations does not violate that policy. Forrest et al. showed that system-call sequences form a "sense of self" for a process [8], which started a long line of host-based anomaly detection [9]. Network datasets such as NSL-KDD have well-known flaws [10,11], and ADFA-LD [12] provides Linux system-call traces of modern attacks.

eBPF security tools.

Falco [13] and Tracee [14] evaluate rules and signatures over kernel events and focus on detection. Tetragon [15] adds in-kernel enforcement, but its response is rule-triggered and binary. Table 1 compares these approaches with KB.

Learning-based response and trust.

Choosing a response under uncertainty is a sequential decision problem suited to reinforcement learning [16]; KB uses PPO [17] with RLlib [18] on Ray [19]. Agreement among agents relates to the Byzantine generals problem [20]. Hash-chained logs [21,22] make tampering detectable, and TPM measured boot [23,24] anchors trust below software, which matters because a software watchdog on a compromised kernel faces the confused-deputy problem [25].

Gaps in existing approaches.

Taken together, the survey leaves eight gaps that KB is designed to close: (L1) signature tools miss novel and obfuscated attacks; (L2) static access-control policy cannot adapt when a permitted binary is hijacked; (L3) user-space monitors can be killed or blinded by a privileged attacker; (L4) most eBPF tools stop at an alert, leaving a human in the critical path; (L5) where enforcement exists it is all-or-nothing, so a false positive on a production database is as damaging as the attack; (L6) nothing prevents a containment pipeline from being steered against PID 1, `systemd` or the defence itself; (L7) plain logs can be edited after the fact; and (L8) systems that automate response rarely supervise the automation itself.

3. Threat Model and Design Goals

We assume an attacker who has gained code execution on the host, for example through a vulnerable network service, and who may escalate to root. The attacker may try to stay within a process's permitted system calls, spread activity across many resources to evade rate limits, flood the telemetry channel, or attack the defence itself. A full kernel compromise, in which the attacker can rewrite kernel memory, is beyond any same-host software defence and is out of scope.

From this model we derive five goals. **G1** enforcement must happen in the kernel; **G2** responses must be proportionate, and reversible below the final level; **G3** no decision path may contain PID 1, kernel threads, critical daemons or KB's own components; **G4** every action must be recorded in a log whose alteration is detectable; and **G5** the decision layer must not start until the layers beneath it have been verified. Table 2 maps the STRIDE categories to the mechanisms that address them.

Table 2. STRIDE analysis of attacks against KB itself

Threat	Example	Mitigation
Spoofing	Rename a process to look like a protected daemon	Gates resolve `/proc/pid/exe`; HASH tier checks SHA-256
Tampering	Edit audit rows to hide an action	Hash chain; verification detects any change
Repudiation	Operator denies an isolation	Every RPC, HTTP action and SSH session is audited
Disclosure	Read the process list over HTTP	Loopback bind, bearer token, origin allow-list
Denial of service	Flood telemetry to stall containment	Per-resource token bucket; separate control socket
Elevation	Use KB against PID 1 or its watchdog	CPM floor and ordered CWP gate

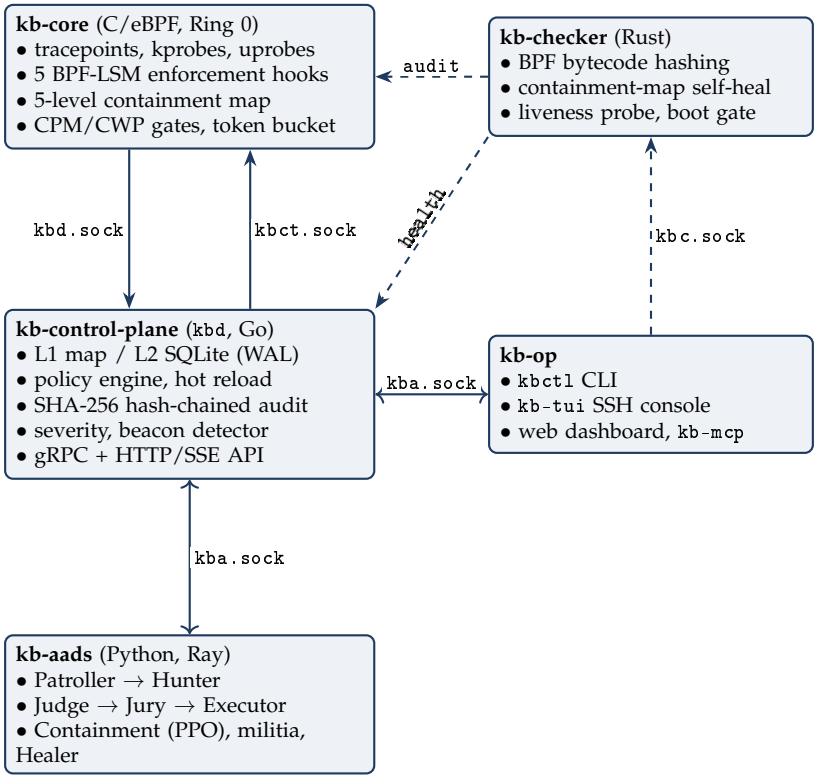


Figure 1. KB dataflow. Solid arrows carry telemetry and commands over root-owned Unix sockets; dashed arrows are the watchdog's audits. Telemetry and control use separate sockets so that a telemetry burst cannot block an enforcement order

4. System Architecture

KB consists of five subsystems (Figure 1). **kb-core** (C/eBPF) is the only Ring-0 component. **kb-control-plane** (`kbd`, Go) ingests telemetry, keeps state, audits and exposes APIs. **kb-aads** (Python/Ray) is the agent swarm. **kb-checker** (Rust) is the integrity watchdog, and **kb-op** provides the operator interfaces.

Telemetry flows one way, from the sensor's user-space bridge to `kbd` over `kbd.sock`; containment, sensitive-path, rule and workload pushes return over a separate `kbct.sock` (Table 3). The split fixes a real availability bug: on a single multiplexed socket, a telemetry burst filled the send buffer, `write()` returned `EAGAIN`, and the sensor treated this as a dead peer, taking containment delivery down with it. All sockets are root-owned Unix domain sockets with mode `0660` and group `kb`; the internal data path has no TCP listener.

Wire contract.

Messages between the C sensor and the Go control plane are packed little-endian frames: a 4-byte length prefix, a 4-byte header (magic `0x4B42`, wire version 3, message type) and a fixed payload. Table 4 lists the message types. Because the two sides are compiled separately, sizes and type codes are

Table 3. Socket topology. All are Unix domain sockets under `/run/kb/`

Socket	Direction	Carries
`kbd.sock`	sensor → `kbd`	`ProcessState` (128 B), `ZoneTransition` (40 B), flow timestamps
`kbct.sock`	`kbd` → sensor	containment (72 B payload), paths, rules, workloads
`kba.sock`	clients ↔ `kbd`	gRPC service (13 RPCs) for swarm and operators
`kbc.sock`	clients ↔ checker	watchdog diagnostics

locked by byte-exact tests on both sides. New types are additive: a receiver ignores a type it does not know, which is how the containment acknowledgement (type 10) was introduced without changing the wire version.

Table 4. Message types on the sensor/control-plane boundary

Type	Message	Direction (socket)	Contents
1	`PROCESS_STATE`	sensor → kbd (`kbd.sock`)	128-byte state: identity, six dimension scores, composite, EMA, entropy, zone
2	`ZONE_TRANSITION`	sensor → kbd (`kbd.sock`)	40 bytes: PID, from/to zone, score, start time
3	rules	kbd → sensor (`kbct.sock`)	compiled behaviour rules pushed at connect time
4	`PROCESS_EXIT`	sensor → kbd (`kbd.sock`)	PID, exit time and code; flushes per-PID state
5	`CONTAINMENT_CMD`	kbd → sensor (`kbct.sock`)	72-byte payload: PID, level 0–4, 64-byte reason
6	`SENSITIVE_PATHS`	kbd → sensor (`kbct.sock`)	policy paths added to the compiled-in floor
7	`CPM_PROTECTED_EXEC`	kbd → sensor (`kbct.sock`)	receiver only; no sender yet (floor is compiled in)
8	`CWP_WORKLOADS`	kbd → sensor (`kbct.sock`)	path, identity tier and SHA-256 per workload
9	`NET_FLOW`	sensor → kbd (`kbd.sock`)	PID, address, port, timestamp for beacon detection
10	`CONTAINMENT_ACK`	sensor → kbd (`kbd.sock`)	PID, level and the gates' decision (Table 7)

Architectural principles.

Three principles shape the design. First, each layer talks only to its neighbours, which limits the blast radius of a failed component. Second, enforcement authority sits in the control plane and the sensor: the swarm is advisory, and disabling it leaves deterministic enforcement by the policy engine running. Third, the watchdog does not trust any component's report about itself; it reads the BPF programs and the kernel containment map directly and compares the map with the control plane's state.

5. Mathematical Formulation

Behaviour score and zones.

Each process has six behavioural dimensions $d_k \in [0, 100]$ (process, syscall, privilege, file, network, memory). An event e sets the dimension it belongs to, $d_{k(e)} \leftarrow v(e)$, where the value depends on the event's flags:

$$v(e) = \begin{cases} 5 & \text{exec or exit (process)} \\ 80 \text{ if escalation, else } 10 & \text{credential change (privilege)} \\ 70 \text{ if sensitive path, else } 5 & \text{file open (file)} \\ 15 & \text{connect or bind (network)} \\ 90 \text{ if RWX, } 25 \text{ if anonymous, else } 5 & \text{mmap or mprotect (memory)} \end{cases} \tag{1}$$

The syscall dimension is not set by individual events. Every ten ring-buffer polls the sensor reads the per-process system-call counters from a kernel map, computes the distribution p_i of the calls made since the previous scan, and normalises its Shannon entropy by the entropy of 512 equally likely calls:

$$d_{\text{sys}} = \min\left(100,\ 100 \cdot \frac{-\sum_i p_i \log_2 p_i}{\log_2 512}\right), \qquad \tilde{d}_{\text{sys},t} = \alpha\, d_{\text{sys},t} + (1-\alpha)\, \tilde{d}_{\text{sys},t-1}, \tag{2}$$

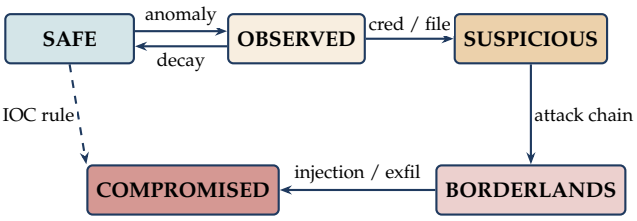


Figure 2. Rule-driven behaviour states. The dashed IOC rule, which would jump directly to COMPROMISED, is designed but cannot yet fire (Section 17)

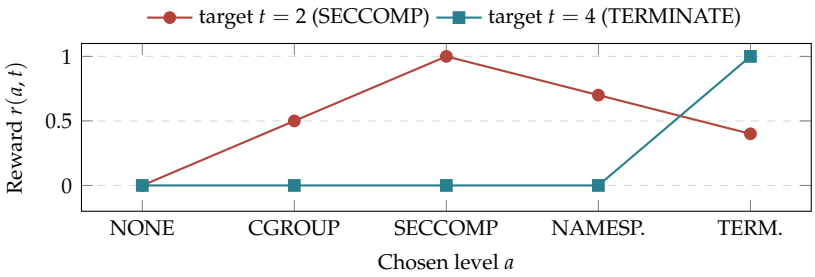


Figure 3. Reward of Eq. 5. Choosing too weak a level is penalised more steeply than choosing too strong a level

and a smoothed value $\tilde{d}_{\text{sys}} \geq 60$ raises a high-entropy evidence flag. The composite score is

$$s_t = \sum_{k=1}^{6} w_k d_k, \tag{3}$$

with $\mathbf{w} = (0.20, 0.25, 0.20, 0.10, 0.10, 0.15)$ in the order above, and it is smoothed by an exponential moving average

$$S_t = \alpha s_t + (1-\alpha) S_{t-1}, \qquad S_0 = s_0, \quad \alpha = 0.3. \tag{4}$$

The zone is SAFE if $S_t < \theta_s$, SUSPICIOUS if $\theta_s \leq S_t < \theta_b$ and BORDERLANDS otherwise, with defaults $\theta_s = 40$, $\theta_b = 75$ that can be overridden per process name. In parallel, a rule engine tracks evidence flags and ordered sequences (for example an RWX mapping followed by a write to process memory within 30 s; all 19 rules are listed in Appendix C) and moves a process from SAFE through OBSERVED, SUSPICIOUS and BORDERLANDS to COMPROMISED.

Containment policy.

Containment is a single-step decision over the observation $o = (\text{score}, \text{zone}, u_{\text{root}}, \Delta\text{score}, e) \in \mathbb{R}^5$ with action $a \in \{0, \ldots, 4\}$ (NONE to TERMINATE). Against the labelled target level t the reward is

$$r(a, t) = 1 - 0.3\,|a - t| - 0.2\,|a - t|\,\mathbb{1}[a < t], \tag{5}$$

so under-containment costs 0.5 per level of error and over-containment 0.3 (Figure 3). The policy π_θ is trained with PPO, which maximises the clipped surrogate

$$L(\theta) = \hat{\mathbb{E}}_t\big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\big], \tag{6}$$

with $\rho_t = \pi_\theta(a_t|o_t) / \pi_{\theta_{\text{old}}}(a_t|o_t)$. Serving is deterministic, $a^* = \arg\max_a \pi_\theta(a \mid o)$.

Jury policy and quorum.

The Jury reuses o with a binary target $t' = \mathbb{1}[t > 0]$ and rewards $+1.0$ (TP), -0.5 (FP), $+0.1$ (TN) and -1.0 (FN). Without a full observation it falls back to a threshold on a confidence $c \in [0, 1]$: CONTAIN if $c > 0.75$, otherwise ALLOW. For votes $\{(v_i, w_i)\}_{i=1}^{n}$ from a freshly spawned Jury pool, containment proceeds iff

$$Q = \frac{\sum_{i:\, v_i = \text{CONTAIN}} w_i}{\sum_{i=1}^{n} w_i} > \frac{1}{2}, \tag{7}$$

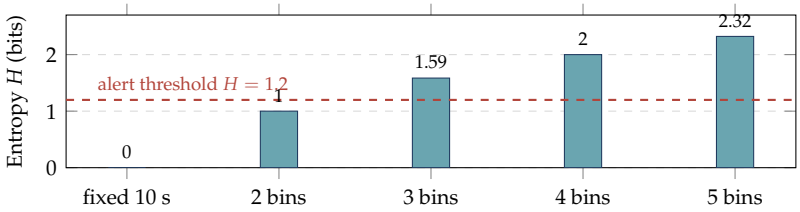


Figure 4. Entropy of Eq. 8 for intervals spread evenly over 1 to 5 bins. Only timing concentrated in at most two bins falls below the alert threshold

and Q is false for $n = 0$. All weights are currently $w_i = 1$, giving a simple majority.

Beaconing and slow exfiltration.

For each (PID, address, port) flow, the last $\min(N, 100)$ inter-connection intervals are binned into $(0, 1], (1, 5], (5, 10], (10, 30], (30, \infty)$ seconds, and

$$H = -\sum_{j=1}^{5} p_j \log_2 p_j \tag{8}$$

is the Shannon entropy [26] of the bin distribution, with a sentinel 3.0 below ten samples. Connection counts x_i per 60 s bucket feed Welford's running mean and variance [27] and a one-sided CUSUM [28] with slack $k = \sigma_i/2$:

$$S_i = \max\bigl(0,\ S_{i-1} + (x_i - \mu_i - k)\bigr), \tag{9}$$

flagged when $S_i > 5\sigma_i$ after at least five buckets. A beacon alert requires both $H < 1.2$ and the CUSUM flag and is debounced to one per flow per five minutes, so neither regular low-volume traffic nor a single burst triggers it alone. Figure 4 shows why the threshold separates beacons from interactive traffic: a fixed-interval beacon puts every interval in one bin ($H = 0$), while timing spread over all five bins reaches the maximum $\log_2 5 \approx 2.32$.

Severity and courthouse oversight.

A score s maps to CRITICAL ($s \geq 90$), HIGH ($s \geq 75$), MEDIUM ($s \geq 50$) or LOW, and is raised one tier if the process matches a registered critical workload; severity routes alerts but never decides containment. For a registered agent with error count e, status σ and uptimes u_0, u_1 read $\tau = 3$ s apart, the verdict is TERMINATE if the agent is unregistered, RESTART if $\sigma \neq \text{active} \vee e \geq 5 \vee u_1 \leq u_0$, and HEALTHY otherwise.

Audit chain.

Each audit row commits to its predecessor,

$$h_i = \mathrm{SHA256}(f_i \,\|\, h_{i-1}), \qquad h_0 = \text{“genesis”}, \tag{10}$$

so altering or deleting a row changes every later hash, which recomputation detects [21].

6. kb-core: Sensor and Enforcement

Hooks and portability.

Tracepoints and kprobes on process exec and exit, system calls, `commit_creds`, `openat`, `connect/bind`, `mmap/mprotect`, `process_vm_writev` and `security_capable`, plus TLS uprobes, write one unified event record into a 1 MB ring buffer. The sensor is built with CO-RE: kernel fields are read only through `BPF_CORE_READ()`, and libbpf relocates them at load time, so one object runs on any kernel from 5.10 with BTF and `CONFIG_BPF_LSM`. Loading is all-or-nothing and depends on the kernel's actual features rather than its version number, which matters because enterprise kernels backport eBPF features: if BTF, a relocation or a program type is missing, the load fails and the sensor exits before

Table 5. kb-core attachment points

Attachment	Type	Purpose
`sched_process_exec/exit`	telemetry	lifecycle; protected-PID registration
`raw_syscalls/sys_enter`	telemetry	per-process syscall counts (entropy)
`kprobe/commit_creds`	telemetry	credential changes (never throttled)
`sys_enter_openat`	telemetry	file access, sensitive paths
`connect` / `bind`	telemetry	outbound flows, listening sockets
`mmap` / `mprotect`	telemetry	anonymous and RWX memory
`process_vm_writev`	telemetry	cross-process memory writes
`kprobe/security_capable`	telemetry	capability probes
`uprobe` TLS write	telemetry	plaintext (OpenSSL, GnuTLS, NSS, Go)
`lsm/file_open`	enforce	level 2: sensitive paths; ≥3: all
`lsm/bprm_check_security`	enforce	level ≥2: deny exec
`lsm/socket_connect, _bind`	enforce	level ≥2: deny IPv4/IPv6
`lsm/file_mprotect`	enforce	level ≥3: deny `PROT_EXEC`

Listing 1: The `file_open` enforcement hook (comments removed).

```
SEC("lsm/file_open")
int BPF_PROG(kb_lsm_file_open, struct file *file, int mask)
{
  __u32 pid = bpf_get_current_pid_tgid() >> 32;
  __u32 *level = bpf_map_lookup_elem(&contained_pids_map, &pid);
  if (level && *level >= 3)
    return -13;                  /* full quarantine */
  char path_buf[64] = {};
  int len = bpf_d_path(&file->f_path, path_buf, sizeof(path_buf));
  if (len < 0) return 0;
  if (len > 63) len = 63;
  #pragma unroll
  for (int i = 63; i >= 0; i--)
    if (i >= len) path_buf[i] = 0; /* exact 64-byte key */
  if (level && *level == 2 && is_sensitive_kernel_path(path_buf))
    return -13;                  /* sensitive paths */
  return 0;
}
```

attaching any program. Table 5 lists the attachment points. Section 11 gives the CO-RE build, the kernel requirements and a compatibility matrix for 16 distributions.

Graded containment.

Five LSM programs (`file_open`, `bprm_check_security`, `socket_connect`, `socket_bind`, `file_mprotect`) consult `contained_pids_map` and deny the operation, returning `-EPERM` from the exec hook and `-EACCES` from the others (Table 6). Level 1 is applied through cgroup v2 limits [29]. Level 4 applies every block the hooks can enforce, and `kbd` then sends `SIGKILL` through a `pidfd`. It does so only if the process passes `kbd`'s own copy of the CPM and CWP checks and the sensor acknowledges that it wrote level 4 to the map; a refusal, or no acknowledgement within 3 s, means no kill. In `file_open`, the full-quarantine check runs *before* `bpf_d_path()`, so a path-resolution failure cannot bypass it. Listing 1 shows the hook with its comments removed. The key is zero-padded because map lookups compare all 64 bytes, and `bpf_d_path()` does not clear the tail.

Authorisation gates.

Any decision layer can be wrong, so two user-space gates run in a fixed order before the kernel map is written (Algorithms 1 and 2). Releasing containment is never gated. The Critical Process Module (CPM) has a compiled-in floor that includes `systemd`, `kbd` and `kb-checker`, so KB cannot be turned against its own watchdog. Critical Workload Protection (CWP) lets an organisation register workloads by path or by SHA-256 hash; a HASH-tier binary that no longer matches its digest is logged as "Identity Verification Failed" and becomes eligible for containment, because it has shown itself not to be the workload it claims to be.

Table 6. Containment levels and what each one blocks

Lvl	Name	Throttle	Sens. files	exec/net	PROT_EXEC	Kill
0	NONE	–	–	–	–	–
1	CGROUP	✓	–	–	–	–
2	SECCOMP	✓	✓	✓	–	–
3	NAMESPACE	✓	✓	✓	✓	–
4	TERMINATE	✓	✓	✓	✓	✓

Algorithm 1: CPM classification (`cpm_classify`)

1: **if** readlink(/proc/pid/exe) fails **then return** REJECT_KTHREAD
2: **if** pid $= 1$ **then return** REJECT_PID1
3: **if** pid $\in$ protected_pids **then return** REJECT_PID
4: path $\leftarrow$ readlink(/proc/pid/exe)
5: **if** path $\in$ protected_exec_paths **then return** REJECT_EXEC
6: **return** ALLOW

Algorithm 2: CWP classification (`cwp_classify`), run only after CPM allows

1: tier $\leftarrow$ lookup(workloads, pid), else by resolved path; **if** none **return** ALLOW
2: **if** tier $=$ PATH **then return** REJECT_PATH
3: **if** path or registry lookup fails **then return** REJECT_HASH {fail closed}
4: **if** SHA256(file) $\neq$ expected **then** log "Identity Verification Failed"; **return** ALLOW
5: **return** REJECT_HASH

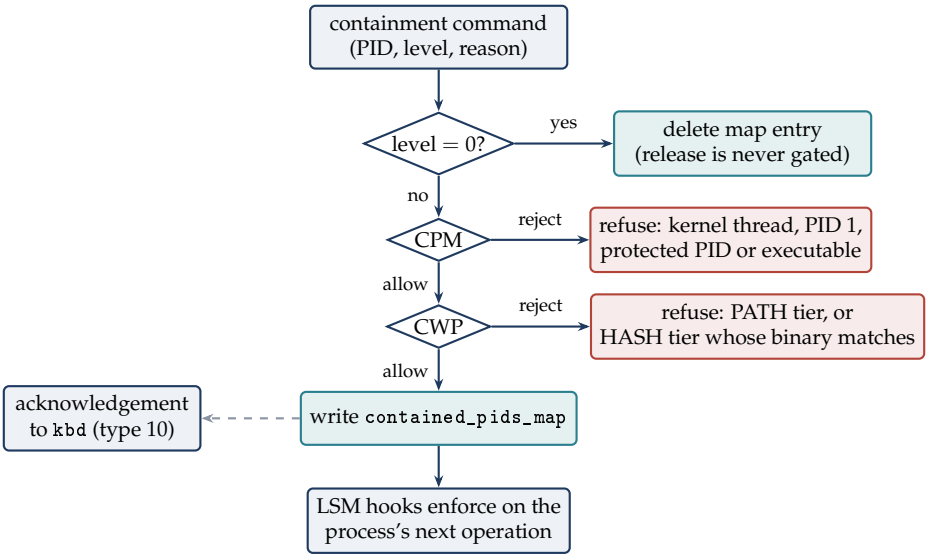


Figure 5. The sensor's handling of a containment command. Every outcome, including refusals and a failed map write, is reported back to `kbd` in a containment acknowledgement (dashed)

Safety of the gates.

Let $\mathcal{P}$ be the set of processes that CPM protects: PID 1, kernel threads, registered PIDs and processes whose resolved executable is on the protected list, which includes `systemd`, `kbd` and `kb-checker`. Let $\mathcal{W}$ be the CWP-registered workloads whose identity check passes (a PATH-tier match, or a HASH-tier binary whose SHA-256 equals its registered digest).

Proposition 1. *For any sequence of containment requests from any decision path (operator, policy engine or swarm), no process in $\mathcal{P} \cup \mathcal{W}$ is written to the containment map at a level above zero, and none is killed by a TERMINATE decision.*

Proof sketch. The containment map is written only by the sensor's command handler, and every request with level $\ell > 0$ passes Algorithm 1 and then Algorithm 2 before the write; a rejection returns without writing. Algorithm 1 rejects every member of $\mathcal{P}$, including when `/proc/pid/exe` cannot be resolved. Algorithm 2 rejects every member of $\mathcal{W}$, and fails closed when the path or registry lookup fails. Requests with $\ell = 0$ only delete entries. TERMINATE's `SIGKILL` is sent only after the sensor acknowledges a successful level-4 write, and through a `pidfd` opened before the request, so it reaches only a process that passed both gates. No other code path writes the map, so the property holds whatever the decision layer requests. It assumes an uncompromised kernel and sensor, which is the threat model of Section 3. □

Rate limiting.

A token bucket (capacity 100, refill one token per 10 ms) is kept per ⟨tgid, start_time, resource⟩ in a 10,240-entry LRU map (Algorithm 3). The start time stops a reused PID inheriting an exhausted bucket, and per-resource keys stop an attacker from spreading activity to stay under a flat per-PID limit. Tier-1 events (credential changes, capability probes, TLS plaintext) are never throttled, and dropped events are reported as a summary count.

TERMINATE and the sensor acknowledgement.

Levels 1–3 are enforced entirely by the hooks. Level 4 adds a `SIGKILL` sent by `kbd`, and because a kill cannot be undone it is guarded twice. Before sending level 4, `kbd` pins the target with `pidfd_open` and applies its own copy of the CPM and CWP rules: PID 1, kernel threads, the protected-executable list (the sensor's floor plus the packaged `/usr/bin` and `/usr/sbin` paths) and the loaded workload registry, failing closed when it cannot classify the process. It then registers for the sensor's reply and sends the command. The sensor answers every containment command with a `CONTAINMENT_ACK` that reports what its own gates and the map write did (Table 7, Figures 5 and 6). `kbd` kills, through the pinned `pidfd`, only on *applied*; a refusal, an unknown code or no reply within 3 s means no kill, and a PID whose request went unanswered is locked out for 30 s so that a late reply cannot answer a newer request. Every non-applied decision is written to the audit log as `CONTAINMENT_REFUSED_BY_SENSOR`. A sensor that predates the acknowledgement never replies, so TERMINATE is refused until the sensor is upgraded.

Table 7. Decision codes in the containment acknowledgement

Code	Decision	Meaning
0	applied	map entry written (level $>$ 0) or removed (level 0)
1	CPM kernel thread	`/proc/pid/exe` could not be resolved
2	CPM PID 1	target is `init`
3	CPM protected PID	PID registered in `protected_pids_map`
4	CPM protected executable	executable on the protected list
5	CWP path	registered PATH-tier workload
6	CWP hash	registered HASH-tier workload whose binary still matches
7	map update failed	gates passed but the BPF map write or delete failed

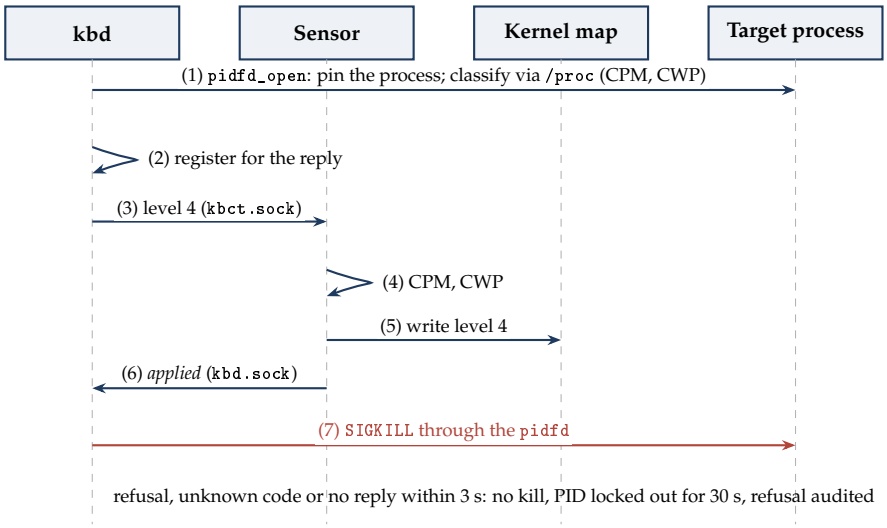


Figure 6. The TERMINATE path. The kill in step 7 happens only after both `kbd`'s own gate (step 1) and the sensor's gates (step 4) allow it and the sensor confirms the map write (step 6)

Algorithm 3: Token-bucket throttle (in eBPF)

key ← ⟨tgid, start_time, resource⟩; b ← lookup or insert $\{C,\ \text{now},\ 0\}$
b.tokens ← $\min(C,\ \text{b.tokens} + \lfloor(\text{now} - \text{b.last})/\Delta\rfloor)$
if b.dropped > 0 **and** b.tokens > 0 **then** emit DROPPED(b.dropped); b.dropped ← 0
if b.tokens > 0 **then** b.tokens−−; **return** ALLOW
b.dropped++; **return** THROTTLE

7. kb-control-plane

`kbd` keeps per-process state in an in-memory L1 `sync.Map`, authoritative for reads, backed by an L2 SQLite database in WAL mode written by a single goroutine through a 4096-entry channel, so ingestion never blocks on disk I/O. On a process exit both layers are flushed, so a reused PID inherits nothing. An upsert preserves any active containment level; an earlier version reset it to NONE on every telemetry update.

Audit log.

Every enforcement action, policy reload and agent decision is appended to the chain of Eq. 10. Verification is exposed through an HTTP route, a gRPC method and `kbctl audit verify`. Our review found that `VerifyChain` did not check `rows.Err()` after iterating, so an I/O error mid-scan looked like a valid, shorter chain. It now returns the error, with a fault-injecting regression test.

Algorithm 4: Audit chain append and verification (`internal/audit`)

LOG(action, subject, actor, reason):
lock; $ts \leftarrow$ now; $c \leftarrow ts \,|\, \text{action} \,|\, \text{subject} \,|\, \text{actor} \,|\, \text{reason} \,|\, \text{prev}$
$h \leftarrow \text{SHA256}(c)$; insert row with (prev, h); prev ← h
VERIFY():
prev ← "genesis"; count ← 0
for each row in id order **do**
    if scan error **then return** (false, count, error)
    if $\text{SHA256}(\text{fields} \,|\, \text{prev}) \neq$ row.hash **then return** (false, count, "chain broken")
    prev ← row.hash; count ← count+1
if the cursor reports an error **then return** (false, count, error) {the fixed defect}
return (true, count, none)

Algorithm 5: Beacon detector, per flow (`internal/detection`)

1: $w \leftarrow$ window[(pid, address, port)] (create if absent)
2: **if** w has a previous timestamp **then** append the interval to w (keep the last 100)
3: **if** the event falls in the current 60 s bucket **then** count $\leftarrow$ count+1
4: **else** fold count into Welford (μ, M_2); $S \leftarrow \max(0, S + \text{count} - \mu - \sigma/2)$; open a new bucket
5: $H \leftarrow$ entropy of the intervals over five bins (3.0 if fewer than 10 samples)
6: cusum $\leftarrow$ (buckets ≥ 5) $\wedge$ $(\sigma > 0)$ $\wedge$ $(S > 5\sigma)$
7: **return** beacon $= (H < 1.2) \wedge$ cusum {alerts debounced to one per flow per 5 min}

Policy and workloads.

`ReloadPolicy` swaps a new policy engine under an `RWMutex` without restarting. The critical-workload registry can be Ed25519-signed; when a public key is configured, a missing or invalid signature is rejected and the last good registry kept. Automatic NAMESPACE containment on a beacon alert is opt-in per process, because a statistical heuristic should not gain enforcement authority by default. Thresholds and enforcement options are set per process name (for example, `nginx` uses 55/85 instead of 40/75), and paths listed in the policy only add to a compiled-in floor of sensitive paths and can never remove it. The shipped defaults are listed in Appendix D.

Internal structure and interfaces.

`kbd` is organised as Go packages for socket I/O and wire decoding (`ipc`), two-layer storage (`store`), per-process policy (`policy`), the audit chain (`audit`), beacon detection (`detection`), translation of decisions into sensor commands (`enforcement`), a client for the watchdog (`checkerclient`) and the orchestrating object with its APIs (`controlplane`). Table 8 lists the two external interfaces. Both are reachable only locally: gRPC on the root-owned `kba.sock`, and HTTP bound to loopback with a CORS allow-list. State-changing HTTP routes require a bearer token compared in constant time; if no token is configured they refuse every request rather than run unauthenticated.

Table 8. External interfaces of `kbd`

Interface	Operations
gRPC `KernelBorderlands` (13 RPCs)	`GetProcessState`, `ListZone`, `SetContainment`, `StreamEvents`, `StreamAlerts`, `SubmitAgentDecision`, `GetSystemStats`, `VerifyAuditChain`, `ExportAuditLog`, `OverrideZone`, `ReloadPolicy`, `ReloadWorkloads`, `RecordSSHSession`
HTTP/SSE (dashboard)	`/api/processes`, `/api/alerts`, `/api/logs`, `/api/audit/verify`, `/api/services`, `/api/events` (SSE), `/api/metrics`, `/api/agents`, `/api/policy`; token-protected: `/api/isolate` (level 3), `/api/kill` (level 4), `/api/restore`, `/api/policy/reload`

8. kb-aads: The Agent Swarm

The swarm is a set of Ray actors sharing a common base class. The **Patroller** dispatches a PID to a **Hunter** pool when its score first crosses 40. The **Judge** opens a consensus round, spawns a fresh **Jury** pool and gates the **Executor** on Eq. 7 (Algorithm 6); a crashed juror is treated as an absent vote, not a veto. The **Containment** agent chooses the level with the trained policy, and the **militia** applies CGROUP $\rightarrow$ SECCOMP $\rightarrow$ NAMESPACE $\rightarrow$ TERMINATE strictly in order, halting at the first failure. The **Healer** is deliberately rule-based: it restores a process only when it is SAFE, scores below 20 and has been contained for 60 ticks, because no ground truth for restore outcomes exists. A `SwarmRegistry` lets the Judge locate agents and apply the courthouse verdict. On a test host the swarm started nine agents with no errors. Table 9 summarises the roles.

Table 9. Agent roles in kb-aads

Agent	Behaviour
Patroller	tracks per-PID scores; hands a PID to a Hunter the first time it crosses the suspicious threshold (40 by default) and re-arms when the score falls back below it
Hunter	investigation of flagged processes; a stub whose evidence-gathering steps are not yet implemented
Judge	opens a consensus round, spawns a fresh Jury pool (5 by default), calls the Executor on quorum, and applies the courthouse check to other agents
Jury	votes CONTAIN or ALLOW with the trained policy given a full observation, otherwise with the 0.75 confidence threshold
Executor	submits a quorum decision to `kbd` over gRPC on `kba.sock`; it never writes the map
Containment	chooses the level 0–4 with the trained PPO policy
Militia	a squad lead spawns one member per stage and applies CGROUP to TERMINATE in order, halting at the first failure
Healer	restores a process only when it is SAFE, scores below 20 and has been contained for 60 ticks
Relays, registry	long-lived actors that route messages to a pool; the registry maps agent IDs to actor handles so the Judge can reach dynamically spawned agents

Integration status.

Every role in Table 9 is implemented and covered by the 68 swarm tests, but release 1.0.0 does not yet connect them into a live decision loop. The Patroller hands a flagged PID to the Hunter, whose investigation is still a stub, and no live code path yet calls the Judge's consensus round, the Containment policy or the militia; the orchestrator leaves the coupling of a quorum decision with a containment level as an open integration step. In the released system, live enforcement therefore comes from the control plane's policy engine and from operators, and the swarm's contribution is the trained policies evaluated in Section 14.

Algorithm 6: Judge consensus round

pool ← spawn n Jury actors (n from configuration); register each
results ← gather(vote(alert) for each juror, return_exceptions)
votes ← results that are not exceptions {crash ⇒ absent vote}
if votes is empty **then return** {round skipped}
if $Q(\text{votes}) > \frac{1}{2}$ **then** Executor.quarantine(alert)

Algorithm 7: Militia escalation (`command_squad`)

stages ← [CGROUP, SECCOMP, NAMESPACE, TERMINATE] truncated to the target level $\ell \in \{1, \ldots, 4\}$
for each stage **do**
    spawn one squad member for the stage and register it
    result ← await member.execute(pid, reason) {stages never race}
    if not result.success **then break** {halt; no retry}
return the stage reports

Algorithm 8: Courthouse check of an agent (`assess_severity`)

h ← registry.lookup(agent); **if** none **then return** TERMINATE
$s_0 \leftarrow h$.status(); **if** s_0.status $\neq$ active **or** s_0.errors ≥ 5 **then return** RESTART
wait $\tau = 3$ s; $s_1 \leftarrow h$.status()
if s_1.uptime $\leq s_0$.uptime **then return** RESTART {agent is not advancing}
return HEALTHY

Training data.

No eBPF-captured attack corpus existed, so all six of ADFA-LD's attack families [12] were mapped to five KB scenarios and target levels, with benign data from ADFA-LD Normal traces and live `/proc`

Table 10. ADFA-LD mapping and labelled rows (6,011 total; split 4,205 / 898 / 908)

ADFA-LD family	KB scenario	Target level	Rows
Normal + live `/proc`	normal	0 NONE	5,265
Hydra-FTP	credential_access	1 CGROUP	162
Hydra-SSH	lateral_movement	2 SECCOMP	176
Adduser	privilege_escalation	3 NAMESPACE	91
Java-Meterpreter	process_injection	3 NAMESPACE	124
Meterpreter, Web-Shell	reverse_shell	4 TERMINATE	193

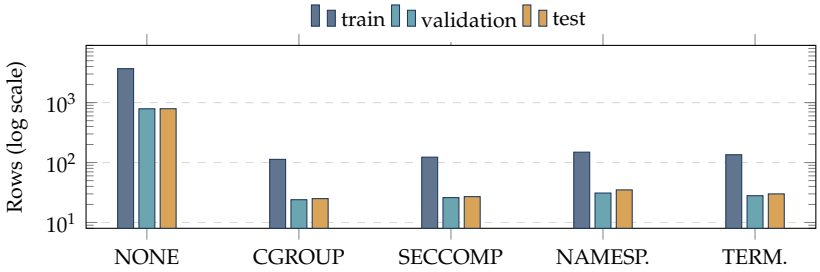


Figure 7. Class distribution of the train, validation and test splits by target level. Benign rows make up 87–88% of each split

telemetry (Table 10, Figure 7). Each row carries the five wire-level features of o. For a trace of length n with syscall diversity $\delta = |\{\text{distinct calls}\}|/n$ and longest run of a repeated call r, each statistic is standardised against the mean and standard deviation of the ADFA-LD Normal traces, and

$$\bar{z} = \tfrac{1}{3}\big(|z_n| + |z_\delta| + |z_r|\big), \qquad \text{score} = \min(100,\, 25\,\bar{z}),$$
$$\Delta\text{score} = \text{clip}_{[-100,100]}\big(25(\bar{z}_2 - \bar{z}_1)\big), \tag{11}$$

where $\bar{z}_1, \bar{z}_2$ are computed on the two halves of the trace. The zone feature is $\max\big(\text{floor}, \min(2, \lfloor \text{score}/40 \rfloor)\big)$, with a per-scenario floor so that a labelled attack is never SAFE. The rows are split 70/15/15 into training, validation and test sets, stratified by class. Training used RLlib PPO with class-balanced sampling (category first, then row), so the rare attack classes receive equal attention.

9. kb-checker and the Boot Sequence

`kb-checker` holds no persistent state and opens no network listener. It hashes the JIT-compiled BPF programs, reconciles the kernel containment map with the control-plane database and heals differences, requires the exec event of a spawned `/bin/true` within 3 s, and flags hooks averaging over 500 ns. When it cannot recover cleanly it escalates from `SIGKILL` of the sensor, to detaching BPF links, to an `iptables` lockdown that keeps only loopback.

In the boot design, order is a security property enforced with plain `systemd` dependencies (Figure 8). `kbd` binds its sockets and signals readiness (`Type=notify`); the watchdog requires it, takes a PID-file lock and runs blocking audits; and the swarm requires the watchdog, so a failed audit prevents the decision layer from starting (goal G5). Release 1.0.0 relaxes this while the watchdog's recovery path is repaired: `kb-checker` is installed but not started by `kbctl up`, and the swarm unit, which is installed but not enabled, depends only on `kbd`. The boot gate is therefore part of the design but not enforced by the packaged units (Section 17).

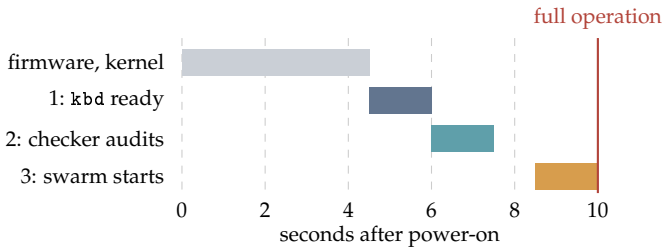


Figure 8. Representative boot timeline of the boot design. Each phase starts only after its `After=/Requires=` dependency is satisfied

Table 11. Audit loops of `kb-checker`

Check	Period	Method
BPF bytecode integrity	60 s	SHA-256 of each loaded program obtained through `bpf_obj_get_info_by_fd`
Control-plane health	5 s	gRPC health check against `kbd`
Swarm health	30 s	status query of the agent swarm
Sensor liveness	60 s	spawn `/bin/true` and require its exec event within 3 s
Containment-map integrity	60 s	compare the kernel map with the control plane's state and heal differences
Hook latency	60 s	warn when a program's average run time exceeds 500 ns

Fallback ladder and failure handling.

Escalation is suppressed for a 15 s start-up grace period, which removed false lockdowns on cold boot. After that, an integrity violation first stops the sensor through `systemctl`; only if that fails does the watchdog kill the sensor process (`pkill -9`), remove the pinned BPF objects under `/sys/fs/bpf`, and set the `iptables` policies to drop all traffic except loopback, so an operator at the console can still intervene. Two other failures are handled by construction. If the sensor cannot load or attach its programs, for example because `bpf` is missing from the active LSM list, it exits before attaching anything and `systemd` restarts it; the host stays up but unprotected, and the failure is visible in the unit status. The watchdog orders itself after the sensor with `Wants=` rather than `Requires=`: a real boot test showed that `Requires=` made a sensor restart issued by the watchdog's own recovery also stop the watchdog, which then misread its own termination as a failed recovery and escalated to a false lockdown.

10. Implementation

Table 12 summarises the implementation, and Appendix E reproduces the code behind the main mechanisms. The sensor is compiled with `clang -target bpf`; `bpftool` generates a skeleton that the user-space loader links against libbpf. The bridge reconnects to `kbd` with exponential back-off between 100 ms and 5 s, so a control-plane restart never requires reloading the hooks. Wire messages are packed little-endian structures whose sizes are locked by byte-exact tests on both the C and Go sides, because a silent mismatch between the two once dropped every containment command. The `KernelBorderlands` gRPC service offers 13 RPCs, including process and zone queries, event and alert streams, `SetContainment`, `SubmitAgentDecision`, audit verification and export, zone override, and policy and workload reload. Generative AI assistance used during development, evaluation and writing is described in the Acknowledgments.

11. Deployment, Portability and Requirements

KB is meant to run on the Linux servers an organisation already has, not on a specially built kernel. This section describes how one sensor binary runs across kernels, which kernel features it needs, how BPF-LSM is enabled, which distributions are supported, and how the packaged release is installed.

Table 12. Implementation by subsystem (non-blank lines of hand-written source and tests; generated code and vendored libraries excluded)

Subsystem	Language	Main components	kLOC
kb-core	C, eBPF	unified sensor, LSM hooks, gates, behaviour engine	6.3
kb-control-plane	Go	ingestion, L1/L2 store, policy, audit, detection, APIs	8.2
kb-aads	Python	Ray actors, JJE consensus, PPO training and inference	2.4
kb-checker	Rust	integrity audits, map self-heal, boot gate, fallbacks	1.1
kb-op	Go, Rust, TS	`kbctl`, `kb-tui`, dashboard, `kb-mcp`	4.2
Total			22.1

11.1. Software Stack

Table 13 lists the software each subsystem needs. Everything is free and open source, and a deployment needs no licences or extra hardware, since KB runs on the server it protects.

Table 13. Software stack by subsystem

Component	**Requirement**
Kernel	Linux 5.10 or later with `CONFIG_DEBUG_INFO_BTF=y`, `CONFIG_BPF_LSM=y` and `bpf` in the active LSM list (Table 14)
eBPF toolchain	clang/LLVM, `bpftool`, vendored libbpf, `libelf` and `zlib`
Control plane	Go 1.25, gRPC and Protocol Buffers, SQLite 3 in WAL mode
Agent swarm	Python 3 with Ray and RLlib ($\geq$2.9) and PyTorch ($\geq$2.0), in a virtual environment
Watchdog and console	Rust (edition 2021); `kb-tui` uses ratatui and a dedicated OpenSSH `sshd`
Dashboard	React 19 and TypeScript, built with Node.js and served by `kbopd`
Service management	`systemd` (`Type=notify`, `After=/Requires=/Wants=`)
Containment primitives	BPF-LSM hooks, cgroup v2 (level 1), `pidfd` signalling (level 4)

11.2. Compile Once, Run Everywhere

An eBPF program reads kernel structures such as `task_struct` and `cred` directly, and their layout changes between kernel versions and vendor builds. KB uses CO-RE [5] so that one compiled object runs on every supported kernel without per-host compilation or a kernel module.

1. *One header for all kernels.* The build generates `vmlinux.h` from the build host's BTF (`bpftool btf dump file /sys/kernel/btf/vmlinux format c`) instead of using version-specific kernel headers.
2. *Relocatable field accesses.* Kernel fields are read only through `BPF_CORE_READ()`: the parent's thread-group ID (`task->real_parent->tgid`) and the new credentials in `commit_creds` (`uid`, `gid`, `euid`, `cap_effective`). Clang (`-g -O2 -target bpf`) records each access as a relocation instead of a fixed offset, and `-g` keeps the BTF that relocation needs.
3. *Fix-up at load time.* When the sensor starts, libbpf matches the relocations against the running kernel's `/sys/kernel/btf/vmlinux` and patches the correct offsets into the bytecode before the verifier sees it.
4. *All-or-nothing load.* If BTF is missing, a relocation cannot be resolved, or the kernel rejects a program or map type, the skeleton load fails and the sensor exits before attaching any program. If loading succeeds but attaching fails, for example because `bpf` is not in the active LSM list, the sensor also exits. A host is therefore never left half-instrumented.

Between load and attach the sensor fills its maps (sensitive paths, the CPM floor and the CWP registry) and registers itself as protected, so no hook runs before protection is in place.

Table 14. Kernel features required by the sensor and the control plane

Feature	Used for	Since Linux
LRU hash map	rate limiter, per-process state	4.10
BTF (`CONFIG_DEBUG_INFO_BTF`)	CO-RE relocation	5.4
`bpf_probe_read_kernel/_user`	reading kernel and user memory	5.5
BPF-LSM (`CONFIG_BPF_LSM`, `bpf` in `lsm=`)	the five enforcement hooks	5.7
BPF ring buffer	the single 1 MB event stream	5.8
`bpf_d_path`	path of the opened file in `lsm/file_open`	5.10
`pidfd_open`, `pidfd_send_signal`	the TERMINATE kill in `kbd`	5.3
cgroup v2	level 1 (CGROUP) throttling	4.5
Minimum kernel for the full feature set		**5.10**

11.3. Enabling BPF-LSM

Telemetry needs only eBPF and BTF, but enforcement needs the BPF Linux Security Module. Two conditions must hold: the kernel is built with `CONFIG_BPF_LSM=y`, and `bpf` appears in the list of active LSMs. Many distributions build the option but leave `bpf` out of the default list, so it must be added to the `lsm=` kernel parameter (Listing 2).

Listing 2: Checking and enabling BPF-LSM

```
ls /sys/kernel/btf/vmlinux                      # BTF present?
cat /sys/kernel/security/lsm                    # is "bpf" in the list?
grep CONFIG_BPF_LSM /boot/config-$(uname -r)    # or: zcat /proc/config.gz (Arch Linux)
# if bpf is missing, append it to the existing list on the kernel command line, e.g.
#   lsm=landlock,lockdown,yama,integrity,apparmor,bpf   and regenerate the boot entry
```

11.4. Distribution Compatibility

Table 15 lists the distributions KB targets. *Native* distributions were tested and deployed with the packaged release and meet every requirement. *Target* distributions ship stock kernels with the required features but have not been validated end to end. *Verify* marks vendor kernels whose BTF or BPF-LSM configuration depends on the exact build; their version numbers understate their eBPF features because enterprise kernels backport them, so the checks of Listing 2 are authoritative. “Boot param” means that BPF-LSM is compiled in but `bpf` must be added to `lsm=`.

Table 15. Linux distribution compatibility

Distribution	Kernel	BTF	BPF-LSM	Status	Notes
Ubuntu 24.04 LTS	6.8	yes	boot param	**Native**	primary development platform
Ubuntu 22.04 LTS	5.15/6.8	yes	boot param	**Native**	`.deb` package
Arch Linux	latest	yes	yes	**Native**	tested and deployed with the Arch package
Debian 12	6.1	yes	boot param	Target	same set-up as Ubuntu
Linux Mint 21/22	5.15/6.8	yes	boot param	Target	Ubuntu LTS base
Kali (rolling)	6.x	yes	boot param	Target	Debian base
Fedora 40+	6.8+	yes	check list	Target	SELinux alongside BPF-LSM
openSUSE Tumbleweed	latest	yes	check list	Target	rolling release
RHEL 9+	5.14+bp	yes	vendor	Verify	backported eBPF features
CentOS Stream 9	5.14+bp	yes	vendor	Verify	upstream of RHEL 9
Rocky / Alma 9+	5.14+bp	yes	vendor	Verify	RHEL-compatible
Oracle Linux 9+	UEK 5.15+	yes	vendor	Verify	prefer UEK with BTF
Amazon Linux 2023	6.1+	yes	check	Verify	verify kernel configuration
Alpine	6.x LTS	check	check	Experimental	musl user space needs static builds
NixOS	config.	config.	config.	Experimental	set `bpf` in kernel parameters
RHEL 7 / Ubuntu 18.04	3.10/4.15	no	no	Unsupported	no BTF, ring buffer or BPF-LSM

11.5. Packaging and Service Units

Release 1.0.0 is built from one install target into an Ubuntu `.deb` and an Arch Linux package, published with SHA-256 checksums and with field manuals for both distributions. Installation provisions the `kb` group, the operator account and the configuration under `/etc/kb`, but starts nothing; `kbctl up` starts the stack. Table 16 lists the `systemd` units as shipped.

Table 16. Service units in release 1.0.0

Unit	Dependencies	Role and status
`kbd.service`	`After=local-fs`	control plane; `Type=notify`; started by `kbctl up`
`kb-sensor.service`	`Requires=kbd`	eBPF sensor; restarted by `systemd` on failure
`kb-checker.service`	`Requires=kbd`, `Wants=kb-sensor`	watchdog; installed, kept down while its recovery path is repaired
`kbagents.service`	`Requires=kbd`	swarm launcher; installed, not enabled
`kbopd.service`	`Requires=kbd`	serves the dashboard
`sshd@kb-operator`	`After=network`	operator SSH on port 2222 with `kb-tui` as forced command

Figure 9 shows how the units, sockets and local network endpoints fit together on one host. All internal sockets live under `/run/kb/`; the only TCP listeners are bound to loopback, apart from the operator's SSH port.

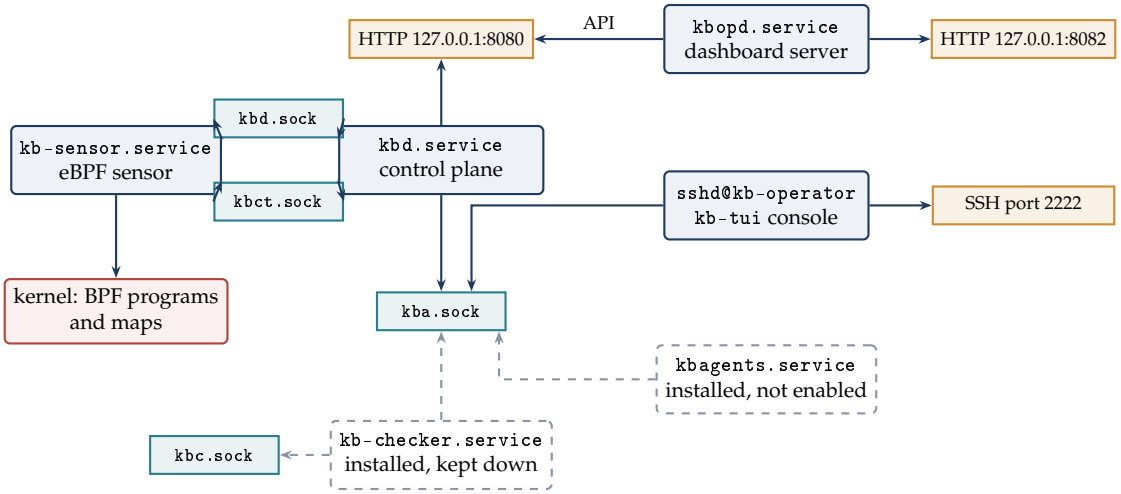


Figure 9. Deployment of release 1.0.0 on one host. Solid boxes are units started by `kbctl up`; dashed boxes are installed but not started. Unix sockets are shown in teal and loopback or operator endpoints in amber

12. End-to-End Containment Path

Figure 10 follows one malicious process from its first kernel event to an enforced and audited containment. The steps cross every trust boundary in the system, and each boundary has a check that does not depend on the layer above it: the verifier for the hooks, the gates for the map write, the LSM hook for the operation, and the hash chain for the record. Steps 6–7 show the designed swarm path; in release 1.0.0 the policy engine or an operator issues the containment request directly (Section 8), as in the live capture of Section 14.

13. Operator Interfaces and Packaging

`kbctl` is a Go gRPC CLI for isolation and restore, zone override, audit verification and export, policy reload, workload signing, and starting or stopping the stack. `kb-tui` is a Rust terminal console reached through a dedicated `sshd` instance on port 2222 with a forced command, so authentication is handled by OpenSSH rather than custom code. The web dashboard requires a bearer token for isolate (level 3, reversible), kill (level 4) and restore, and `kb-mcp` exposes a read-only statistics tool (event rate

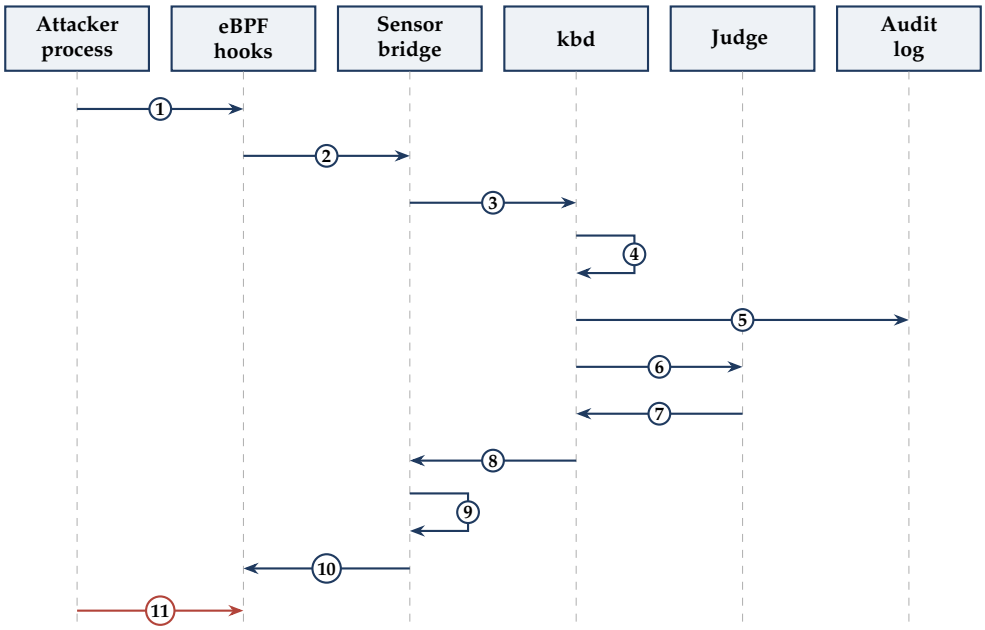


Figure 10. End-to-end containment path. (1) exec, credential, file, network and memory events; (2) rate-limited ring buffer; (3) `ProcessState` over `kbd.sock`; (4) score and zone reach BORDERLANDS; (5) hash-chained audit append; (6) alert over `kba.sock`; (7) quorum reached, `SetContainment`; (8) command over `kbct.sock`; (9) CPM then CWP gates; (10) containment map written; (11) the next blocked operation is denied. Steps 1–5 run without the swarm, and the policy engine can issue step 8 directly

and number of active processes) to LLM tooling over the Model Context Protocol. KB is distributed as an Ubuntu `.deb` and an Arch Linux package built from one install target; the packages provision the `kb` group and the operator account but start nothing until an operator runs `kbctl up`.

Table 17. Operator interfaces

Interface	Functions
`kbctl` (Go, gRPC)	`process isolate -level` (default NAMESPACE), `zone override`, `audit verify|export`, `policy reload`, `workload reload|keygen|sign`, `stats`; stack control `up|down|status`; `dashboard up|down|status|url|open`; `ssh up|down|status|connect|add-key`; `tui`
`kb-tui` (Rust, ratatui)	four views: Processes, Alerts, Agent Activity and Query Console, with confirmed containment actions; reached over `sshd@kb-operator` on port 2222 with a forced command, and every session is recorded in the audit chain
`kb-dashboard` (React, TypeScript)	Processes, Zone Topology, Alerts, Telemetry, Services, Containment, Consensus Log, Rogue Management, Console and Settings; Isolate and a two-step Kill are separate actions
`kb-mcp` (Go)	Model Context Protocol server over stdio with one read-only tool, `kb.get_statistics`

14. Evaluation

14.1. Containment and Jury Policies

The first PPO run reached 97.8% overall test accuracy but only 60.0% on CGROUP and 70.4% on SECCOMP. The cause was in the features, not the model: two attack scenarios shared one engineered event-type code and zone floor. Giving each scenario its own code lifted every attack class to 97.1–100% and cut under-containment from 1.1% to 0.0% (Figure 11). Overall accuracy fell to 84.4% because the benign class, 87% of the test set, fell from 100% to 82.2%. We accept this trade deliberately: a false positive costs a review or a restore, while a false negative costs a compromise, which is exactly the asymmetry of Eq. 5.

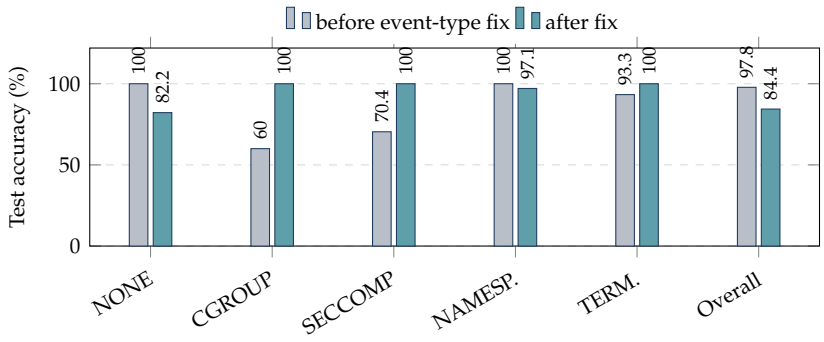


Figure 11. Containment policy accuracy per level on 908 held-out samples. Attack classes rise to 97.1–100%; under-containment falls from 1.1% to 0.0%

Table 18. Jury policy on the 908-sample test set

	Pred. C	Pred. A	Metric	Value
Actual CONTAIN	117	0	Accuracy	100.0%
Actual ALLOW	0	791	Recall (C)	100.0%
			FN rate	0.0%

The Jury collapses the same labels to CONTAIN/ALLOW and uses the same splits. It reached 100.0% validation and test accuracy with zero false negatives (Table 18). Training is not seeded, so figures vary slightly between runs (an earlier run scored 99.8% and 99.4%), but zero false negatives held in every run observed. The binary task is easier because the attack classes that the five-way policy must separate collapse into one label.

14.2. Live Detection and Response

We captured telemetry from a live run on an Ubuntu virtual machine during a controlled attack simulation: 188 rounds at a 3 s interval, each touching the file, memory and network dimensions of one long-lived process. The capture holds 10,440 sensor events (7,703 hook events), 489 zone transitions and 409 audit rows. The attack process escalated SAFE → SUSPICIOUS → BORDERLANDS six times, and no other process reached BORDERLANDS. Each time, the control plane issued a CGROUP throttle within about 1 ms of the BORDERLANDS audit record, and the sensor logged it as applied 0.53–1.90 s later, with a mean of 1.35 s (Figure 12). These delays come from journal timestamps and so include logging latency.

Privilege-change events dominate the hook stream (51.4%), and the sensor flagged 3,563 hook events (46%) as sensitive (Figure 13). The ring buffer reported overload 1,863 times at 101–270 events/s, with 40,761 events dropped in total, which is the load that the rate limiter and the Tier-1 exemption are designed for. Half of the watchdog's 855 log lines are health checks and a quarter are recovery actions, reflecting ten sensor restarts during the capture. These are feasibility traces rather than a detection-rate benchmark: one attack script was used, the benign baseline is short, and watchdog-forced restarts leave gaps in the sensor stream.

14.3. Functional Scenarios

Table 19 summarises the live scenarios run against a running sensor and control plane. The three attack chains exercise the rule engine end to end, and the gate scenarios confirm that protection holds even while the telemetry channel is saturated.

14.4. Test Case Study

Testing ran at four levels because each level caught defects the others missed: unit tests (Go with in-memory SQLite, pytest with injectable fake Jury and militia classes so that consensus logic runs without a Ray cluster, `cargo test`, and C tests for the behaviour engine); byte-exact wire tests on both sides of the C/Go boundary; live-sensor scripts that drive real hooks (Table 19) and verify CPM and

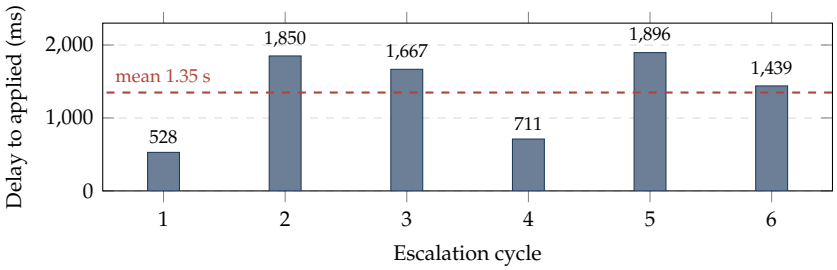


Figure 12. Delay from each BORDERLANDS transition to the containment action logged by the sensor

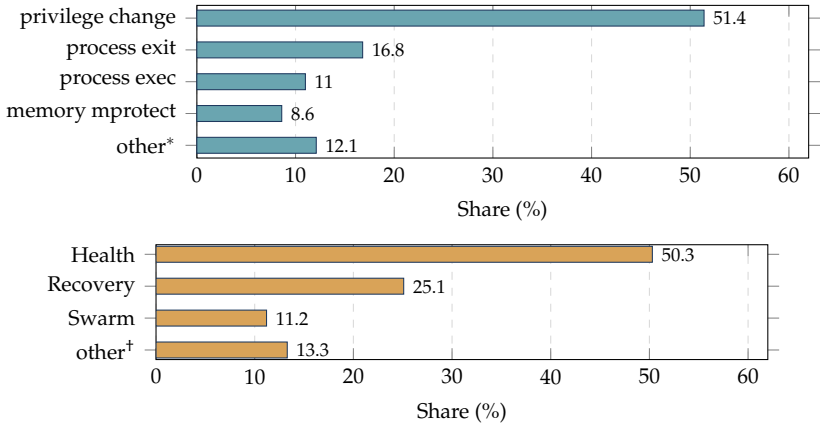


Figure 13. Composition of the live capture. Top: hook events by type (7,703). Bottom: watchdog log lines by tag (855). *File access, network connect/bind, mmap, TLS plaintext. †Performance, map audit, integrity, warnings, liveness

CWP against a running sensor; and boot tests of the unit chain. Tables 20–22 list representative cases by area, each backed by a named automated test or live script in the repository; all pass on release 1.0.0.

Table 20. Test cases: containment, gates and the kill path

ID	Input	Expected and observed
C1	contain PID 1, or `kbd`'s own PID	refused before any command is sent
C2	contain a process named `systemd` or `kbd`; TERMINATE on `kb-checker`	refused by `kbd`'s name floor; refused by its executable list
C3	contain a protected PID while `kbd.sock` is flooded (live)	refused; control path unaffected
C4	CWP PATH-tier workload; modified HASH-tier binary (live)	refused; "Identity Verification Failed" and allowed
C5	routine telemetry for a contained PID	containment level preserved (regression test)
C6	restore (level 0) on a protected PID	allowed: release is never gated
C7	dashboard Isolate	sent as NAMESPACE, never TERMINATE
C8	TERMINATE refused by `kbd`'s gate	nothing sent, nothing killed
C9	TERMINATE refused by the sensor, or no acknowledgement	no kill; unanswered PID locked out
C10	acknowledgement for a different PID	ignored
C11	levels 1–3	no kill and no wait for an acknowledgement
C12	real `pidfd` against a test child process	child killed; PID 1 refused

Table 19. Live functional scenarios; all passed

Scenario	Observed behaviour
All telemetry types (exec, exit, creds, files, connect, bind, mmap, mprotect)	expected events, flags and sensitive-path marks
Chain A: privilege change + credential access	SAFE → OBSERVED → SUSPICIOUS
Chain B: memory injection	SUSPICIOUS → BORDERLANDS → COMPROMISED
Chain C: C2 connection to port 4444	SAFE → SUSPICIOUS → BORDERLANDS
Contain PID 1, `kbd` or `kb-checker`	refused by CPM
Contain a protected PID while `kbd.sock` is flooded	refused; control path unaffected
CWP PATH tier / modified HASH-tier binary	refused / "Identity Verification Failed", allowed
Level 2 process runs `curl`; level 3 calls `mprotect(PROT_EXEC)`	denied in both cases (`EPERM` on exec, `EACCES` on connect and `mprotect`)
Tampered audit row; I/O error during verification	chain reported broken; error surfaced
Rigid 10 s beacon vs. human-like timing	$H \approx 0$ flagged; high entropy ignored

Table 21. Test cases: audit chain, API security, workloads and detection

ID	Input	Expected and observed
A1	modify one row's hash, then verify	chain reported broken
A2	fault-injecting driver fails the cursor mid-scan	error surfaced, not a shorter valid chain
A3	state-changing HTTP route with no server token configured	503: fails closed
A4	wrong or missing bearer token	401
A5	request from an unknown origin	CORS header not reflected
A6	tampered, unsigned or wrong-key workload registry	rejected; previous registry kept
A7	HASH-tier workload with the hash omitted	digest computed locally from the binary
A8	rigid beacon vs. varied, human-like timing	low entropy flagged; high entropy ignored
A9	low entropy without a volume shift	no beacon verdict: both signals required
A10	sustained rigid beacon	one alert per flow per five minutes
A11	BORDERLANDS alert for a CWP workload	severity raised one tier with owner metadata

Table 22. Test cases: agent swarm

ID	Input	Expected and observed
S1	Jury confidence exactly 0.75; just above	ALLOW (strict threshold); CONTAIN
S2	tied votes; empty vote list	no quorum; false without raising
S3	two of three jurors crash, survivor votes CONTAIN	round completes, quarantine executed
S4	all jurors crash	round skipped without raising
S5	militia target level 3; NAMESPACE stage fails at target 4	three stages in order; halts before TERMINATE
S6	agent uptime does not advance in 3 s	courthouse verdict RESTART
S7	Healer with a missing field	keep contained
S8	Jury with a full observation and the trained checkpoint	policy used; threshold fallback without it
S9	Patroller score crossing the threshold twice	escalated once until the score drops

14.5. Test Suites and Implementation Size

Testing ran at four levels, because each level caught defects the others missed: unit tests, byte-exact wire-contract tests, live-sensor scenarios and end-to-end boot tests of the unit chain (Figure 14). The control plane has 164 Go tests and the swarm 68 pytest tests, with injectable fake actors so consensus logic is testable without a Ray cluster. The live script triggers all event types and three multi-stage attack chains against a running sensor; separate scripts verify CPM and CWP against it. The hand-written implementation, excluding generated code and vendored libbpf, is about 22,000 lines.

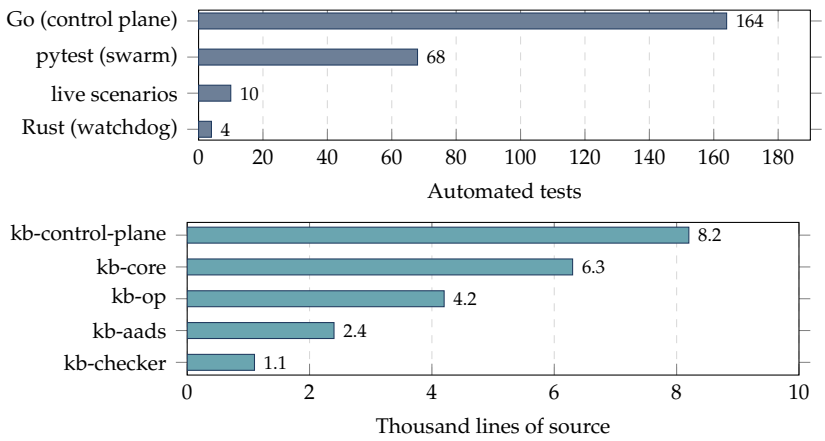


Figure 14. Test coverage (top) and implementation size (bottom). "live scenarios" counts the scenarios of the live-sensor script; generated and vendored code excluded

Table 23. Median cost per operation, all hooks loaded (5 runs each)

Workload	KB off	KB on	Added	Change
null system call (`getppid`)	67.1 ns	105.2 ns	38 ns	+57%
`open` + `close`	425 ns	624 ns	199 ns	+47%
UDP socket + `connect`	1.38 μs	1.67 μs	290 ns	+21%
`mmap`/`mprotect`	959 ns	1,109 ns	150 ns	+16%
`fork` + `exec /bin/true`	225 μs	252 μs	26 μs	+11.7%
Compile (`make -j4`)	1.007 s	1.010 s	–	+0.4%
`sysbench cpu`, 4 threads	15,588/s	15,473/s	–	−0.7%

14.6. Run-Time Overhead

We measured overhead on a workstation with an Intel Core Ultra 9 275HX (24 threads) running Linux 7.2 with BTF, BPF-LSM and kprobes enabled. The installed `kbd` and sensor services ran with all 18 sensor programs loaded: 5 BPF-LSM hooks, 2 kprobes, 2 uprobes and 9 tracepoints, including `raw_syscalls/sys_enter`. Each workload ran five times, alternating between KB stopped and KB running, with the CPU governor fixed to `performance` and the state database reset before each KB run; we report medians (Table 23, Figure 15).

The added cost is a roughly fixed amount per hooked operation: 38 ns on every system call, from the `sys_enter` hook that feeds the syscall-entropy dimension, a few hundred nanoseconds for file, network and memory operations that have their own hooks, and 26 μs per process creation. In relative terms this is large for a null system call but disappears in real work: a parallel compile and a CPU-bound benchmark both changed by less than 1%, within run-to-run noise. Resident memory was about 28 MB for `kbd` and 12 MB for the sensor. The micro-benchmarks are single-process floods, so the sensor's token bucket (burst 100, refilled at 100 per second per process, start time and resource) dropped and counted 150,000–190,000 events per run, which is its designed behaviour. On an earlier 4-vCPU virtual machine where the LSM and kprobe programs could not be loaded, the absolute per-operation costs were about twice as high, and a compile slowed by 14%; the difference between hosts shows that overhead should be measured on the deployment hardware. Aggregating `sys_enter` counts in a kernel map instead of per event would remove most of the remaining per-syscall cost.

Every individual run is listed in Appendix F.

14.7. Threats to Validity

The policy results come from ADFA-LD traces mapped to KB's features, not from KB's own sensor, so they show that the feature design and reward separate the attack classes, not that the deployed system detects those attacks at the same rate. The test split shares the generation process of the training data, and each of the four attack levels has only 25–35 test samples, so per-level accuracy has wide uncertainty. The live capture used a single attack script on one virtual machine, and the full-hook

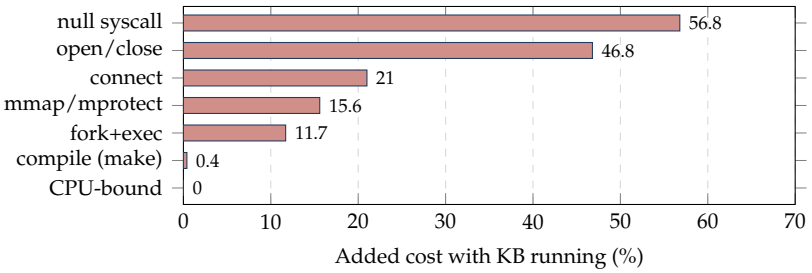


Figure 15. Overhead by workload with all 18 sensor programs loaded. The cost is a fixed amount per hooked operation, so it matters only for operation-intensive code

Table 24. Cost of the main operations (P processes, A audit rows, n jurors)

Operation	Cost	Note
Telemetry hook, token bucket	$O(1)$	map lookups, ring-buffer reserve
LSM enforcement hook	$O(1)$	`file_open` adds path resolution
Score update	$O(1)$	six dimensions
L1 upsert / L2 write	$O(1)$ / $O(\log P)$	L2 is off the hot path
Audit append / verify	$O(1)$ / $O(A)$	one SHA-256 per row
Beacon observe	$O(1)$	$\leq$100 intervals per flow
CWP HASH tier	$O(\text{file})$	only on a containment decision
Jury round	$O(n)$	parallel; latency of slowest juror

overhead was measured on a single host and kernel. A labelled eBPF corpus and an independent benchmark are needed before the detection rate can be stated.

15. Discussion

Cost per event.

Every kernel-side operation is constant time and bounded by the verifier (Table 24), so the per-event cost does not grow with load (measured in §14), and all kernel maps are sized at load time so a hostile process cannot exhaust them. User-space state grows with the number of live processes and is flushed on exit; only the audit log grows without bound, by design.

Memory footprint.

Table 25 lists the fixed and growing structures. All kernel-side memory is fixed when the maps are created, so a hostile process cannot exhaust it, and the LRU maps age out stale entries. User-space state scales with live processes and is flushed on exit. Only two structures grow without bound: the audit log, which is append-only so that history cannot be lost, and the per-flow detector windows, which still need time-based eviction for idle flows.

Table 25. Memory footprint of the main data structures (P processes, A audit rows, F active flows, $W \leq 100$ intervals)

Structure	**Size**	**Growth**
`kb_events` ring buffer	1 MB	fixed; shared by all hooks
rate-limiter LRU map	10,240 entries	fixed; LRU eviction
per-process system-call counts	655,360 entries	fixed (10,240 processes $\times$ 64)
`contained_pids_map`	1,024 entries	fixed
`protected_pids_map`	2,048 entries	fixed
`protected_workloads_map`	8,192 entries	fixed
protected executable and workload paths	64 entries each	fixed; 64-byte keys
user-space scoring table	10,240 slots	fixed
L1 `sync.Map` and L2 process rows	$O(P)$	bounded by live processes; flushed on exit
L2 audit log	$O(A)$	append-only by design
beacon-detector windows	$O(F \cdot W)$	at most 100 intervals per flow; no idle eviction yet

Design choices.

Several deliberate choices trade generality for assurance. Unix sockets instead of TCP keep the internal data path off the network. SQLite in WAL mode with a single writer needs no listener and removes lock contention, while the L1 map absorbs reads. `systemd` dependencies make the boot gate exactly as trustworthy as `systemd` itself and easy for operators to inspect. The CPM and CWP checks run in the sensor's user-space bridge rather than in eBPF, because hashing a binary and resolving `/proc/pid/exe` cannot be done safely in the kernel; the cost is a few microseconds per containment decision, which is rare compared with telemetry. Containment is a single-step decision because that is how the control plane asks the question (which level for this process now?), and it keeps training stable on a small dataset. The Healer stays rule-based, and beacon-triggered containment stays opt-in, because neither has ground truth that would justify giving a learned or statistical rule enforcement authority.

Learned decisions under a deterministic floor.

The swarm is advisory: CPM and CWP sit below every decision path, the policy engine can enforce without the swarm, and the reward shape encodes the asymmetry between a false positive and a missed attack. This layering means that a mistaken or compromised decision layer can at worst act against an unprotected process: the Healer or an operator can lift levels 1–3, and every action, including a TERMINATE kill, is recorded in the audit log.

Lessons.

Four lessons from building KB generalise beyond it. *Live tests find what unit tests cannot*: the most serious defects, namely mismatched wire constants that silently dropped every containment command, a socket design that let telemetry bursts kill the control path, and a missing readiness notification that froze the boot chain, all passed their unit tests and were found only on a running system. *Order checks deliberately and fail closed*: checking full quarantine before resolving a path, rejecting a HASH-tier containment when the path cannot be resolved, and surfacing a cursor error decide whether a security property actually holds. *Look at per-class metrics*: a 97.8%-accurate model was failing on two of five classes, and the fix was in feature engineering, not in the learning algorithm. *Claims decay*: re-checking earlier audit findings, and this paper's own claims, against the current source repeatedly found statements that had become out of date.

16. Security Audit and Defects Found

We re-verified every previously reported open defect directly against current source rather than carrying old claims forward. Most were already fixed: an unauthenticated HTTP API with wildcard CORS (now a constant-time bearer token and an origin allow-list), the telemetry update that reset containment, the LSM fail-open ordering, a custom SSH server without throttling (replaced by OpenSSH), a hard-coded dashboard host, unvalidated containment RPCs and three Jury consensus defects. Only the `rows.Err()` gap in the audit verifier was genuinely open, and it is now fixed. Audit findings decay as code moves and must be rechecked at every revision.

Integration testing on real hosts found defects that every unit test had passed (Table 26). Several would have disabled a security property while reporting success.

17. Limitations and Future Work

We state the limitations plainly, because a security system is defined as much by what it does not do as by what it does.

- *Training data.* The policies are trained on ADFA-LD, a public stand-in: it covers no memory-exploit scenario, and there is no mapping yet from the sensor's live event stream to the training features. An attack lab that records labelled eBPF telemetry is the next step, followed by a detection-rate benchmark.

Table 26. Defects found by live and real-host testing

Defect	Fix
Go and C disagreed on the containment message type (3 vs. 5); every command was dropped while the audit log recorded success	Constants aligned; wire contract locked by tests
Telemetry burst on a shared socket killed the control path	Separate `kbd.sock` and `kbct.sock`; `SIGPIPE` ignored
Unconditional sensitive-path block broke `sudo`, `su` and login host-wide	Block only for contained processes
`bpf_d_path()` left the key tail unzeroed, so exact matches never hit	Explicit zero padding of the 64-byte key
`kbd` never sent `sd_notify`; the unit chain stayed "activating"	Poll sockets, then notify READY
Watchdog locked the host down on every cold boot	15 s start-up grace period
Unscored PID's zero-value `uid=0` read as root	Empty process name means not found
Packaged operator `sshd` lacked the capabilities for privilege separation, and the operator account was locked	Minimal capability set; account unlocked without a usable password

- *Swarm integration.* The swarm's roles are implemented and tested, but in release 1.0.0 no live path yet connects a flagged process to the consensus round, the Containment policy and the militia (Section 8). The Hunter is a stub planned as a fine-tuned, tool-calling language model; the Healer needs restore-outcome labels before it can be learned; and the courthouse reaches only the restart verdict until peer-drift and boundary-violation detectors exist.
- *Watchdog and boot gate.* The watchdog's recovery path still causes sensor restarts under load, so release 1.0.0 keeps it down and the swarm unit no longer requires it; the boot gate of Section 9 is therefore not enforced by the packaged units until the escalation logic is hardened.
- *Kill under load.* The sensor's acknowledgement shares the telemetry socket, so under a telemetry flood it can be lost; TERMINATE then fails closed and leaves the process contained but alive. Only TERMINATE waits for the acknowledgement; for levels 1–3 a sensor refusal is audited but not returned to the caller.
- *Overhead.* Overhead was measured on one workstation and kernel with synthetic workloads; it should be repeated on server hardware with a production-like service.
- *Detection inputs.* The beacon detector uses connection frequency as a proxy for byte volume, because `connect()` telemetry carries no payload size; detector windows and the rate limiter keep state in memory, and idle flows are not yet evicted.
- *Behaviour rules.* Seven of the 19 shipped rules need evidence flags or sequence steps the sensor does not yet produce (shell spawn, `ptrace`, anonymous executable mappings, exec from `/tmp`, connect bursts), so they cannot fire (Appendix C).
- *Coverage and scope.* Operator additions to the CPM executable list have a receiver in the sensor but no sender yet, so the floor is compiled in. `kb-mcp` has not been audited. All policy is host-local, with no fleet management.
- *Trust domain.* Any same-host defence shares the kernel's trust domain; a full kernel compromise is beyond it.

Future work follows directly from these limits:

1. an attack lab that captures labelled eBPF telemetry from scripted attacks in isolated virtual machines, so the policies are retrained on KB's own observation surface;
2. a live decision loop that couples the Jury's quorum with the Containment policy and the militia, with the Hunter built as a tool-calling agent using `kb-mcp` as its interface;
3. restore-outcome labels for a learned Healer, and drift and boundary detectors for the full courthouse;
4. multi-step escalation episodes in which the policy observes the effect of each stage;
5. byte-level exfiltration telemetry from `sendmsg/write` accounting;

6. server benchmarks (for example web, database and kernel-build workloads) and a hardened watchdog that restores the boot gate;
7. fleet operation with a signed central policy store and aggregated audit chains; and
8. moving the last line of defence to an out-of-band hardware attestation module, which is being pursued as separate work.

18. Conclusion

Kernel Borderlands shows that a Linux runtime defence can sit where attacks actually happen, inside the kernel, while keeping every automated decision proportionate, gated, auditable and supervised. Its contribution is the combination: graded BPF-LSM enforcement behind authorisation gates that the decision layer cannot bypass, a hash-chained control plane, a swarm whose containment and jury decisions are learned and checked by quorum, and a watchdog that gates the boot sequence. The policies reach 97–100% accuracy on every attack class with no under-containment and no Jury false negatives on held-out data, and the live capture shows the complete loop from kernel event to enforced, audited containment.

Author Contributions: P.S.R.V.K. led the project, designed the system architecture, implemented the kernel subsystem (kb-core: the eBPF sensor, BPF-LSM enforcement and the CPM and CWP gates) and the integrity watchdog (kb-checker), wrote the documentation, and drafted the manuscript. T.K. implemented the control-plane subsystem (kb-control-plane: the `kbd` daemon with its gRPC and IPC interfaces, storage, process lifecycle, HTTP and SSE API, enforcement and audit chain). K.K. implemented the agent-swarm subsystem (kb-aads: the Ray agents, consensus, dataset pipeline and reinforcement-learning training) and carried out the attack simulation and adversarial testing. R.Y.K. implemented the operator-interface subsystem (kb-op: the command-line tool, the SSH terminal console, the web dashboard and the MCP server). Each author contributed the manuscript sections on their own subsystem. All authors contributed to testing, reviewed the manuscript, and approved the final version.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: ADFA-LD is publicly available from UNSW Canberra. The labelled training, validation and test splits used in this study are in the code repository (https://github.com/pardhuvarmax/kernel-borderlands, directory `scripts/dataset/output`). The source code, release packages and field manuals are in the same repository. Other data are available from the corresponding author on reasonable request.

Acknowledgments: We thank the maintainers of the Linux kernel, libbpf, Ray and RLlib, and the authors of the ADFA-LD dataset, on whose work this project builds. The research, architecture, design and core implementation are the authors' own work. Under the authors' direction, generative AI (Anthropic Claude, via Claude Code) was used for module-level and repository-wide testing including edge cases, the run-time benchmark, field documentation from the authors' data, minor bug fixes and selected implementation tasks recorded in the project's change log, and editing and typesetting this manuscript. The authors reviewed and verified all AI-assisted output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AADS	Agent-Assisted Defence Swarm
BTF	BPF Type Format
CO-RE	Compile Once, Run Everywhere
CPM	Critical Process Module
CUSUM	Cumulative sum (control chart)
CWP	Critical Workload Protection
eBPF	extended Berkeley Packet Filter
EMA	Exponential moving average
JJE	Judge–Jury–Executor
LRU	Least recently used
LSM	Linux Security Module
MCP	Model Context Protocol
PPO	Proximal Policy Optimisation
RPC	Remote procedure call
SSE	Server-sent events
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
WAL	Write-ahead logging

Appendix A. Training configuration

Both policies were trained with the same configuration (Table A1), taken from the training scripts in the repository. Training ran on CPU; other settings are RLlib defaults.

Table A1. Training configuration of the Containment and Jury policies

Setting	Value
Algorithm	PPO, RLlib (new API stack), PyTorch
Learning rate	5×10^{-4}
Training batch / mini-batch	2,000 / 256 samples
Iterations / epochs per iteration	40 / 10
Episode	single step; reward of Eq. 5 (Containment) or TP/FP/TN/FN rewards (Jury)
Observation	score, zone, root flag, score delta, event type
Sampling	class-balanced: category first, then row
Data split	4,205 / 898 / 908 rows (70/15/15, stratified, seed 42)
Training seed	not seeded

Appendix B. Reproducing the Results

All artefacts are in the public repository (https://github.com/pardhuvarmax/kernel-borderlands), and this paper describes release 1.0.0.

- *Packages.* The `dist/` directory contains the Ubuntu package `kernel-borderlands_1.0.0_amd64.deb`, the Arch Linux package `kernel-borderlands-1.0.0-1-x86_64.pkg.tar.zst`, a `SHA256SUMS` file and field manuals for both distributions. Install with `apt install ./<file>.deb` or `pacman -U <file>.pkg.tar.zst`, then run `sudo kbctl up`.
- *Build from source.* `make` in `kb-core/` regenerates `vmlinux.h` and builds the sensor; `go build ./...` in `kb-control-plane/` builds the control plane.
- *Data.* The labelled training, validation and test splits are in `scripts/dataset/output/`; the dataset pipeline and training scripts are in `kb-aads/`.
- *Tests.* `go test ./...` in `kb-control-plane/` (164 tests), `pytest` in `kb-aads/` (68 tests), `cargo test` in `kb-checker/`, and `kb-core/tests/test_all_hooks.sh` against a running sensor.
- *Kernel check.* Run the commands of Listing 2 before loading the sensor.

Appendix C. Behaviour Rules and Evidence Flags

The behaviour engine moves a process between states when a rule's conditions hold (Section 5). Release 1.0.0 ships 19 rules in `rules.yaml`, compiled by `kbd` and pushed to the sensor when it connects (Table A2). A rule fires when all its required flags are set, at least the stated number of its optional flags are set, and, where given, its sequence occurs in order within the window. The sensor currently sets 13 of the defined evidence flags (Table A3); seven rules depend on flags or sequence steps that are defined but not yet produced, so they cannot fire. They are kept because they encode the intended detections and will start working as the corresponding telemetry is added.

Table A2. Behaviour rules shipped in release 1.0.0 (states abbreviated: OBS observed, SUSP suspicious, BORD borderlands, COMP compromised)

Rule	Transition	Conditions (flags; optional flags; sequence)	Window	Can fire
known_ioc_sequence	SAFE→COMP	OUTBOUND_CONNECT, RWX_MAPPING; EXEC_FROM_TMP, EXEC_FROM_PROC (≥1); seq. EXEC → OUTBOUND_CONNECT → RWX_MAP	30 s	**no**
reverse_shell_compromised	BORD→COMP	OUTBOUND_CONNECT, SPAWNED_SHELL; seq. OUTBOUND_CONNECT → EXEC_SHELL	60 s	**no**
injection_ptrace_compromised	BORD→COMP	RWX_MAPPING, PTRACE_USED; seq. RWX_MAP → PTRACE	30 s	**no**
injection_proc_mem_compromised	BORD→COMP	RWX_MAPPING, PROC_MEM_WRITE; seq. RWX_MAP → PROC_MEM_WRITE	30 s	yes
escalation_exfil_compromised	BORD→COMP	PRIVILEGE_GAINED, OUTBOUND_CONNECT; SHADOW_ACCESS, PASSWD_ACCESS, SSH_KEY_ACCESS (≥1)	120 s	yes
rwx_memory_abuse	SUSP→BORD	RWX_MAPPING, PROC_MEM_WRITE, PROCESS_VM_WRITE (≥1)	–	yes
ptrace_injection_borderlands	SUSP→BORD	PTRACE_USED	–	**no**
c2_port_connection	SUSP→BORD	OUTBOUND_CONNECT, C2_CANDIDATE_PORT	–	yes
proc_mem_write_borderlands	SUSP→BORD	PROC_MEM_WRITE	–	yes
rapid_connect_burst_borderlands	SUSP→BORD	RAPID_CONNECT_BURST	–	**no**
multi_anomaly_suspicious	OBS→SUSP	EXEC_FROM_TMP, EXEC_FROM_PROC, SPAWNED_SHELL, PRIVILEGE_GAINED, ROOT_ACHIEVED, RWX_MAPPING, ANON_EXEC, WX_TRANSITION, OUTBOUND_CONNECT, NONSTANDARD_PORT, SHADOW_ACCESS, SSH_KEY_ACCESS, HIGH_SYSCALL_ENTROPY (≥3)	–	yes
privilege_plus_network_suspicious	OBS→SUSP	PRIVILEGE_GAINED, OUTBOUND_CONNECT	60 s	yes
cred_file_access_suspicious	OBS→SUSP	SHADOW_ACCESS, PASSWD_ACCESS, SUDOERS_ACCESS, SSH_KEY_ACCESS (≥1)	–	yes
anon_exec_mapping_suspicious	OBS→SUSP	ANON_EXEC	–	**no**
shell_spawn_suspicious	OBS→SUSP	SPAWNED_SHELL	–	**no**
unexpected_outbound_connection	SAFE→OBS	OUTBOUND_CONNECT	–	yes
privilege_change_observed	SAFE→OBS	PRIVILEGE_GAINED, ROOT_ACHIEVED, CAP_GAINED (≥1)	–	yes
high_syscall_entropy_observed	SAFE→OBS	HIGH_SYSCALL_ENTROPY	–	yes
first_anomaly_observed	SAFE→OBS	any flag	–	yes

Table A3. Evidence flags by group; flags marked ∗ are set by the release 1.0.0 sensor

Group	Flags
Execution	`EXEC_FROM_TMP`, `EXEC_FROM_PROC`, `SPAWNED_SHELL`, `ANOMALOUS_PARENT`, `RAPID_EXEC_BURST`, `EXECVEAT`
Privilege	`PRIVILEGE_GAINED`∗, `ROOT_ACHIEVED`∗, `CAP_GAINED`∗, `SETUID_ABUSE`
Memory	`RWX_MAPPING`∗, `ANON_EXEC`, `WX_TRANSITION`, `LARGE_ANON_EXEC`, `PROC_MEM_WRITE`∗, `PROCESS_VM_WRITE`
Files	`SHADOW_ACCESS`∗, `PASSWD_ACCESS`∗, `SUDOERS_ACCESS`∗, `SSH_KEY_ACCESS`∗, `CRON_WRITE`, `BINARY_PATH_WRITE`
Network	`OUTBOUND_CONNECT`∗, `NONSTANDARD_PORT`, `C2_CANDIDATE_PORT`∗, `BIND_LISTENER`∗, `DNS_TUNNEL_SUSPECT`, `RAPID_CONNECT_BURST`
System calls	`HIGH_SYSCALL_ENTROPY`∗, `PTRACE_USED`, `SECCOMP_BYPASS_ATTEMPT`, `UNUSUAL_SYSCALL_SEQ`

Appendix D. Default Configuration

The package installs its configuration under `/etc/kb`. Listings 3–6 show the shipped defaults with comments removed. The workload registry ships empty; its documented example registers a database by path and a secrets manager by hash. Note that the default policy enables automatic termination for `postgres`, which operators should review now that TERMINATE kills.

Listing 3: Default policy (`policy.yaml`)

```yaml
defaults:
  suspicious:   40.0
  borderlands:  75.0

sensitive_paths:
  - /tmp/kb-pipeline-test-secret.txt

policies:
  - comm: nginx
    suspicious:   55.0
    borderlands:  85.0
    allow_network: true
    auto_terminate: false

  - comm: postgres
    suspicious:   50.0
    borderlands:  80.0
    allow_network: false
    auto_terminate: true

  - comm: bash
    suspicious:   45.0
    borderlands:  70.0
    auto_terminate: false
```

Listing 4: Swarm composition (`agents.yaml`)

```yaml
ray:
  mode: local

swarm:
  patroller: 2
  hunter: 2
  healer: 1
  containment: 1
  signal_relay: 1

jury:
  pool_size: 5

patroller:
  suspicious_threshold: 40.0

control_plane:
  grpc_socket: /run/kb/kba.sock
```

Listing 5: Documented workload-registry example (`workloads.yaml`; the shipped file is empty)

```yaml
critical_workloads:
  - path: /usr/bin/postgres
    identity_tier: path
    owner_team: data-platform
    justification: "Primary production OLTP database"
  - path: /usr/bin/vault
    identity_tier: hash
    owner_team: security-infra
    justification: "Secrets management; elevated identity verification required"
```

Listing 6: One behaviour rule from `rules.yaml`

```yaml
  - name: injection_proc_mem_compromised
    description: "RWX memory allocation followed by writing to proc memory"
```

```
    required_flags:
      - KB_EV_RWX_MAPPING
      - KB_EV_PROC_MEM_WRITE
    sequence:
      - KB_SEQ_RWX_MAP
      - KB_SEQ_PROC_MEM_WRITE
    window_seconds: 30
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_INJECTION_CHAIN
    min_source_state: KB_STATE_BORDERLANDS
```

Appendix E. Selected Source Listings

The listings below are taken from release 1.0.0 with comments removed; elisions are marked. They show the code behind the main mechanisms: the in-kernel rate limiter (Listing 7), an enforcement hook (Listing 8), the sensor's gated command handler (Listing 9), the control plane's containment path with the TERMINATE kill (Listing 10), the audit chain (Listing 11), the Jury round (Listing 12) and the watchdog's liveness check (Listing 13). In the rate limiter, setting `last_time` to the current time after a refill discards the fractional part of the elapsed interval, so the effective refill rate is slightly below 100 tokens per second.

Listing 7: Token-bucket rate limiter (`kbd_sensor.bpf.c`)

```
static __always_inline int kb_rate_limit_throttle(__u64 resource_id)
{
    struct task_struct *task = (struct task_struct *)bpf_get_current_task();
    struct kb_token_bucket *bucket = NULL;

    struct kb_lru_key key = {};
    key.tgid = BPF_CORE_READ(task, tgid);
    key.start_time = BPF_CORE_READ(task, start_time);
    key.resource_id = resource_id;

    bucket = bpf_map_lookup_elem(&kb_rate_limit_lru_map, &key);
    if (!bucket) {
        struct kb_token_bucket new_bucket = {};
        new_bucket.tokens = KB_RL_MAX_TOKENS;
        new_bucket.last_time = bpf_ktime_get_ns();
        bpf_map_update_elem(&kb_rate_limit_lru_map, &key, &new_bucket, BPF_ANY);
        bucket = bpf_map_lookup_elem(&kb_rate_limit_lru_map, &key);
        if (!bucket) return 0;
    }

    __u64 now = bpf_ktime_get_ns();
    __u64 elapsed = now - bucket->last_time;

    if (elapsed > KB_RL_REFILL_RATE_NS) {
        __u64 add_tokens = elapsed / KB_RL_REFILL_RATE_NS;
        bucket->tokens += add_tokens;
        if (bucket->tokens > KB_RL_MAX_TOKENS) {
            bucket->tokens = KB_RL_MAX_TOKENS;
        }
        bucket->last_time = now;
    }

    if (bucket->dropped_count > 0 && bucket->tokens > 0) {
         struct kb_unified_event *e = bpf_ringbuf_reserve(&kb_events, sizeof(*e), 0);
         if (e) {
             __builtin_memset(e, 0, sizeof(*e));
             e->pid = BPF_CORE_READ(task, tgid);
             e->event_type = KB_EVT_DROPPED_TELEMETRY;
             e->length = bucket->dropped_count;
```

```
            e->ts_ns = now;
            bpf_ringbuf_submit(e, 0);
        }
        bucket->dropped_count = 0;
    }

    if (bucket->tokens > 0) {
        bucket->tokens--;
        return 0;
    } else {
        bucket->dropped_count++;
        return 1;
    }
}
```

Listing 8: Connect enforcement hook (`kbd_sensor.bpf.c`)

```
SEC("lsm/socket_connect")
int BPF_PROG(kb_lsm_socket_connect, struct socket *sock,
             struct sockaddr *address, int addrlen)
{
    __u32 pid = bpf_get_current_pid_tgid() >> 32;
    __u32 *level = bpf_map_lookup_elem(&contained_pids_map, &pid);
    if (!level)
        return 0;
    if (*level < 2)
        return 0;

    __u16 family = 0;
    bpf_probe_read_kernel(&family, sizeof(family), &address->sa_family);
    if (family == 2  || family == 10 )
        return -13;
    return 0;
}
```

Listing 9: Gated containment command handler with acknowledgement (`kbd_sensor.c`; logging trimmed)

```
static void handle_incoming_containment_cmd(int fd, struct kbd_sensor_bpf *skel)
{
    /* length-prefixed frame read and header validation omitted */
    uint32_t pid   = cmd->pid;
    uint32_t level = cmd->level;
    int map_fd = bpf_map__fd(skel->maps.contained_pids_map);
    if (map_fd < 0) {
        send_containment_ack(pid, level, KB_ACK_MAP_UPDATE_FAILED);
        return;
    }
    if (map_fd >= 0) {
        if (level == 0) {

            if (bpf_map_delete_elem(map_fd, &pid) != 0 && errno != ENOENT) {
                fprintf(stderr, "[SENSOR] delete_elem failed for PID %u: %s\n", pid, strerror(errno));
                send_containment_ack(pid, level, KB_ACK_MAP_UPDATE_FAILED);
            } else {
                printf("[SENSOR] Cleared containment for PID %u (Reason: %.64s)\n", pid, cmd->reason);
                send_containment_ack(pid, level, KB_ACK_APPLIED);
            }
        } else {

            enum cpm_decision decision = cpm_classify(skel, (pid_t)pid);
            if (decision != CPM_ALLOW) {
                send_containment_ack(pid, level, cpm_decision_to_ack(decision));
```

```
                return;
            }

            enum cwp_decision wdecision = cwp_classify(skel, (pid_t)pid);
            if (wdecision != CWP_ALLOW) {
                send_containment_ack(pid, level, cwp_decision_to_ack(wdecision));
                return;
            }
            if (bpf_map_update_elem(map_fd, &pid, &level, BPF_ANY) != 0) {
                fprintf(stderr, "[SENSOR] update_elem failed for PID %u with level %u: %s\n", pid,
     level, strerror(errno));
                send_containment_ack(pid, level, KB_ACK_MAP_UPDATE_FAILED);
            } else {
                printf("[SENSOR] Applied containment level %u to PID %u (Reason: %.64s)\n", level, pid,
      cmd->reason);
                send_containment_ack(pid, level, KB_ACK_APPLIED);
            }
        }
    }
}
```

Listing 10: Containment and the TERMINATE kill (`internal/enforcement/enforce.go`)

```
func (e *Enforcer) Contain(pid uint32, level uint32, reason string) error {

        if level != ipc.ContainmentNone {
                if pid == 1 {
                        return fmt.Errorf("enforcement: refusing to contain pid 1 (init)")
                }
                if pid == uint32(os.Getpid()) {
                        return fmt.Errorf("enforcement: refusing to contain self (kbd, pid=%d)", pid)
                }
                if e.store != nil {
                        if cs, ok := e.store.GetProcessState(pid); ok && e.protectedComm[cs.Comm] {
                                return fmt.Errorf("enforcement: refusing to contain protected process %
     q (pid=%d)", cs.Comm, pid)
                        }
                }
        }

        switch level {
        case ipc.ContainmentNone, ipc.ContainmentCgroup, ipc.ContainmentSeccomp,
                ipc.ContainmentNamespace, ipc.ContainmentTerminate:

                var target terminateTarget
                var ackCh <-chan uint32
                var cancelAck func()
                if level == ipc.ContainmentTerminate {
                        t, err := prepareTerminate(e, pid)
                        if err != nil {
                                return err
                        }
                        defer t.Close()
                        target = t

                        ackCh, cancelAck, err = e.acks.expect(pid, level)
                        if err != nil {
                                return err
                        }
                        defer cancelAck()
                }
                if err := e.listener.SendContainmentCmd(pid, level, reason); err != nil {
                        return fmt.Errorf("enforcement: failed to send containment cmd: %w", err)
                }
```

```
                if target != nil {
                        select {
                        case decision := <-ackCh:
                                if decision != ipc.ContainmentAckApplied {
                                        return fmt.Errorf("enforcement: sensor refused terminate of pid %d (%s); not killing", pid, ipc.AckDecisionString(decision))
                                }
                        case <-time.After(ackTimeout):
                                e.acks.markUnresolved(pid)
                                return fmt.Errorf("enforcement: sensor did not acknowledge terminate of pid %d within %s; not killing (fail closed)", pid, ackTimeout)
                        }
                        if err := target.Kill(); err != nil && !errors.Is(err, syscall.ESRCH) {
                                return fmt.Errorf("enforcement: terminate acknowledged but SIGKILL to pid %d failed: %w", pid, err)
                        }
                }
                return nil
        default:
                return fmt.Errorf("enforcement: unknown containment level %d", level)
        }
}
```

Listing 11: Audit chain append and verification (`internal/audit/audit.go`)

```
func (a *Logger) Log(action, subject, actor, reason string) error {
    a.mu.Lock()
    defer a.mu.Unlock()

    ts := time.Now().UnixNano()
    content := fmt.Sprintf("%d|%s|%s|%s|%s|%s",
        ts, action, subject, actor, reason, a.prevHash)
    hash := fmt.Sprintf("%x", sha256.Sum256([]byte(content)))

    _, err := a.db.Exec(`
        INSERT INTO audit_log (ts_ns,action,subject,actor,reason,prev_hash,entry_hash)
        VALUES (?,?,?,?,?,?,?)
    `, ts, action, subject, actor, reason, a.prevHash, hash)
    if err != nil { return err }

    log.Printf("[AUDIT] %s | %s | %s...", action, subject, hash[:16])
    a.prevHash = hash
    return nil
}

func (a *Logger) VerifyChain() (bool, int, error) {
    rows, err := a.db.Query(`
        SELECT ts_ns,action,subject,actor,reason,prev_hash,entry_hash
        FROM audit_log ORDER BY id ASC
    `)
    if err != nil { return false, 0, err }
    defer rows.Close()

    prev, count := "genesis", 0
    for rows.Next() {
        var ts int64
        var action, subject, actor, reason, prevH, entryH string
        if err := rows.Scan(&ts, &action, &subject, &actor, &reason, &prevH, &entryH); err != nil {
            return false, count, err
        }

        content := fmt.Sprintf("%d|%s|%s|%s|%s|%s",
            ts, action, subject, actor, reason, prev)
```

```
        expected := fmt.Sprintf("%x", sha256.Sum256([]byte(content)))
        if expected != entryH {
            return false, count, fmt.Errorf("chain broken at entry %d", count+1)
        }
        prev = entryH
        count++
    }

    if err := rows.Err(); err != nil {
        return false, count, err
    }
    return true, count, nil
}
```

Listing 12: Quorum test and Jury round (`consensus/jje.py`)

```
def quorum_reached(votes: list) -> bool:
    if not votes:
        return False
    contain_votes = sum(v["weight"] for v in votes if v["vote"] == "CONTAIN")
    total_votes = sum(v["weight"] for v in votes)
    return contain_votes / total_votes > 0.5

async def coordinate_consensus(self, alert_payload: dict):

        self._round_counter += 1
        jury_pool = [self.jury_agent_cls.remote(f"jury-{i}") for i in range(self.jury_pool_size)]
        if self.registry is not None:

            for i, jury in enumerate(jury_pool):
                self.registry.register.remote(f"jury-round{self._round_counter}-{i}", AgentRole.JURY.
     value, jury)

        vote_futures = [jury.evaluate_and_vote.remote(alert_payload) for jury in jury_pool]
        results = await asyncio.gather(*vote_futures, return_exceptions=True)
        votes = []
        for result in results:
            if isinstance(result, Exception):
                print(f"[JudgeAgent] jury vote failed, excluding from quorum: {result}")
            else:
                votes.append(result)

        if not votes:
            print("[JudgeAgent] no jury votes collected -- skipping consensus round")
            return

        if quorum_reached(votes):

            await self.executor.execute_quarantine.remote(alert_payload)
```

Listing 13: Sensor liveness check (`kb-checker/src/integrity/mod.rs`; logging trimmed)

```
pub async fn verify_liveness(uds_path: &str) -> Result<(), Box<dyn std::error::Error>> {
    let path = uds_path.to_string();
    let channel = Endpoint::try_from("http://[::]:50051")?
        .connect_with_connector(service_fn(move |_| {
            let path_clone = path.clone();
            async move {
                UnixStream::connect(path_clone).await
            }
        }))
        .await?;
    let mut client = KernelBorderlandsClient::new(channel);

```

```
    let filter = EventFilter {
        event_types: vec![],
    };

    let mut response_stream = client.stream_events(filter).await?.into_inner();

    let start_time = std::time::Instant::now();
    let mut child = std::process::Command::new("/bin/true").spawn()?;
    let child_pid = child.id();

    let mut detected = false;
    let timeout_dur = Duration::from_secs(3);

    while let Ok(res) = tokio::time::timeout(timeout_dur, response_stream.message()).await {
        match res {
            Ok(Some(event)) => {
                if event.pid == child_pid || event.comm == "true" {
                    detected = true;
                    break;
                }
            }
            _ => break,
        }
        if start_time.elapsed() > timeout_dur {
            break;
        }
    }

    let _ = child.kill();

    if detected {
        Ok(())
    } else {
        Err("BPF hook liveness check failed: Heartbeat process exec event not intercepted in stream.".
     into())
    }
}
```

Appendix F. Raw Run-Time Overhead Data

Table A4 gives every run behind the medians of Table 23. Baseline and KB runs alternated five times on the same host (Intel Core Ultra 9 275HX, 24 logical CPUs, Linux 7.2.11 with the `performance` governor, active LSMs `capability,landlock,lockdown,yama,bpf`), with the sensor and `kbd` built from commit `450ff3e` and their SHA-256 digests recorded. Resident memory stayed between 27.3 and 29.9 MB for `kbd` and 12.0 and 12.5 MB for the sensor. Idle CPU over 30 s was 0.3% of one core for each process in the first three runs, and 0.8–1.2% (`kbd`) and 5.0–8.4% (sensor) in the last two runs. The rate limiter dropped between 151,579 and 189,881 events per run during the floods.

Table A4. All five alternating runs (base: no KB; KB: all 18 programs loaded)

Benchmark		Run 1	Run 2	Run 3	Run 4	Run 5	Median
Null system call (ns)	base	72.4	65.7	65.4	67.1	76.2	67.1
	KB	98.9	106.5	105.2	105.3	99.4	105.2
open/close (ns)	base	424	431	511	422	425	425
	KB	624	762	611	617	633	624
socket connect (ns)	base	1293	1312	1447	1380	1387	1380
	KB	1670	1565	1647	1701	1796	1670
mmap/mprotect (ns)	base	959	1189	930	901	978	959
	KB	1107	1109	1075	1154	1232	1109
fork+exec (μs)	base	225.4	222.4	220.2	230.9	234.5	225.4
	KB	243.7	251.7	248.0	259.8	266.2	251.7
compile, `make -j4` (s)	base	0.999	0.983	1.082	1.007	1.124	1.007
	KB	0.987	1.010	0.988	1.012	1.087	1.010
`sysbench cpu` (events/s)	base	15,013	15,588	15,737	15,698	14,530	15,588
	KB	15,698	15,153	15,633	15,473	14,762	15,473

Appendix G. Wire Structures

Listing 14 gives the packed C structures behind the message types of Table 4, taken from `kb-core/userspace/bridge/` with comments removed. Every frame is preceded by a 4-byte little-endian length. The Go side decodes the same layouts field by field, and tests on both sides fix the sizes: 128 bytes for `kb_wire_process_state`, 40 bytes for `kb_wire_zone_transition`, 76 bytes (a 72-byte payload after the header) for the containment command, and 16 bytes for the acknowledgement.

Listing 14: Packed wire structures (`kb_bridge.h` and `kb_bridge.c`)

```c
#pragma pack(push, 1)
struct kb_wire_header {            /* 4 bytes */
    uint16_t magic;                /* 0x4B42, "KB" */
    uint8_t  version;              /* 3 */
    uint8_t  msg_type;             /* Table 4 */
};
struct kb_wire_process_state {     /* type 1, 128 bytes */
    struct kb_wire_header hdr;
    uint32_t pid;
    uint32_t ppid;
    uint32_t uid;
    char     comm[16];
    uint64_t start_time_ns;
    uint64_t last_updated_ns;
    double   dim_score[KB_DIM_COUNT];   /* six dimensions */
    double   composite_score;
    double   ema_score;
    double   syscall_entropy_lifetime;
    uint32_t zone;
    uint32_t event_count;
};
struct kb_wire_zone_transition {   /* type 2, 40 bytes */
    struct kb_wire_header hdr;
    uint32_t pid;
    uint64_t start_time_ns;
    uint32_t from_zone;
    uint32_t to_zone;
    double   score;
    uint64_t ts_ns;
};
struct kb_wire_containment_cmd {   /* type 5, 76 bytes */
    struct kb_wire_header hdr;
```

```
    uint32_t pid;
    uint32_t level;                /* 0 NONE .. 4 TERMINATE */
    char reason[64];
};
struct kb_wire_containment_ack {   /* type 10, 16 bytes */
    struct kb_wire_header hdr;
    uint32_t pid;
    uint32_t level;
    uint32_t decision;             /* Table 8 */
};
struct kb_wire_net_flow {          /* type 9, 24 bytes */
    struct kb_wire_header hdr;
    uint32_t pid;
    uint32_t daddr;
    uint16_t dport;
    uint16_t _reserved;
    uint64_t ts_ns;
};
#pragma pack(pop)
```

References

1. Gregg, B. *BPF Performance Tools: Linux System and Application Observability*; Addison-Wesley: Boston, 2019.
2. Singh, K. Kernel Runtime Security Instrumentation (KRSI) / BPF LSM. Linux kernel documentation, https://docs.kernel.org/bpf/prog_lsm.html, 2020.
3. McCanne, S.; Jacobson, V. The BSD Packet Filter: A New Architecture for User-level Packet Capture. In Proceedings of the Proceedings of the USENIX Winter 1993 Conference, 1993, pp. 259–270.
4. Høiland-Jørgensen, T.; Brouer, J.D.; Borkmann, D.; Fastabend, J.; Herbert, T.; Ahern, D.; Miller, D. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In Proceedings of the Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT), 2018, pp. 54–66. https://doi.org/10.1145/3281411.3281443.
5. libbpf authors. libbpf and BPF CO-RE (Compile Once – Run Everywhere). https://github.com/libbpf/libbpf, 2026. Accessed 27 October 2026.
6. Loscocco, P.; Smalley, S. Integrating Flexible Support for Security Policies into the Linux Operating System. In Proceedings of the Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 2001, pp. 29–42.
7. Wright, C.; Cowan, C.; Smalley, S.; Morris, J.; Kroah-Hartman, G. Linux Security Modules: General Security Support for the Linux Kernel. In Proceedings of the Proceedings of the 11th USENIX Security Symposium, 2002, pp. 17–31.
8. Forrest, S.; Hofmeyr, S.A.; Somayaji, A.; Longstaff, T.A. A Sense of Self for Unix Processes. In Proceedings of the Proceedings of the IEEE Symposium on Security and Privacy, 1996, pp. 120–128. https://doi.org/10.1109/SECPRI.1996.502675.
9. Liao, H.J.; Lin, C.H.R.; Lin, Y.C.; Tung, K.Y. Intrusion Detection System: A Comprehensive Review. *Journal of Network and Computer Applications* **2013**, *36*, 16–24. https://doi.org/10.1016/j.jnca.2012.09.004.
10. Tavallaee, M.; Bagheri, E.; Lu, W.; Ghorbani, A.A. A Detailed Analysis of the KDD CUP 99 Data Set. In Proceedings of the Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009, pp. 1–6. https://doi.org/10.1109/CISDA.2009.5356528.
11. McHugh, J. Testing Intrusion Detection Systems: A Critique of the 1998 and 1999 DARPA Intrusion Detection System Evaluations as Performed by Lincoln Laboratory. *ACM Transactions on Information and System Security* **2000**, *3*, 262–294. https://doi.org/10.1145/382912.382923.
12. Creech, G.; Hu, J. Generation of a New IDS Test Dataset: Time to Retire the KDD Collection. In Proceedings of the Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC), 2013, pp. 4487–4492. https://doi.org/10.1109/WCNC.2013.6555301.
13. The Falco Project. Falco: Cloud Native Runtime Security. https://falco.org/, 2026. Accessed 27 October 2026.
14. Aqua Security. Tracee: Runtime Security and Forensics Using eBPF. https://github.com/aquasecurity/tracee, 2026. Accessed 27 October 2026.

15. Cilium Authors. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/, 2026. Accessed 27 October 2026.
16. Nguyen, T.T.; Reddi, V.J. Deep Reinforcement Learning for Cyber Security. *IEEE Transactions on Neural Networks and Learning Systems* **2023**, *34*, 3779–3795. https://doi.org/10.1109/TNNLS.2021.3121870.
17. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* **2017**.
18. Liang, E.; Liaw, R.; Nishihara, R.; Moritz, P.; Fox, R.; Goldberg, K.; Gonzalez, J.; Jordan, M.; Stoica, I. RLlib: Abstractions for Distributed Reinforcement Learning. In Proceedings of the Proceedings of the 35th International Conference on Machine Learning (ICML), 2018, pp. 3053–3062.
19. Moritz, P.; Nishihara, R.; Wang, S.; Tumanov, A.; Liaw, R.; Liang, E.; Elibol, M.; Yang, Z.; Paul, W.; Jordan, M.I.; et al. Ray: A Distributed Framework for Emerging AI Applications. In Proceedings of the Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018, pp. 561–577.
20. Lamport, L.; Shostak, R.; Pease, M. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* **1982**, *4*, 382–401. https://doi.org/10.1145/357172.357176.
21. Schneier, B.; Kelsey, J. Secure Audit Logs to Support Computer Forensics. *ACM Transactions on Information and System Security* **1999**, *2*, 159–176. https://doi.org/10.1145/317087.317089.
22. Crosby, S.A.; Wallach, D.S. Efficient Data Structures for Tamper-Evident Logging. In Proceedings of the Proceedings of the 18th USENIX Security Symposium, 2009, pp. 317–334.
23. Trusted Computing Group. Trusted Platform Module Library, Family 2.0, Level 00, Revision 1.59. Technical report, Trusted Computing Group, 2019.
24. Sailer, R.; Zhang, X.; Jaeger, T.; van Doorn, L. Design and Implementation of a TCG-based Integrity Measurement Architecture. In Proceedings of the Proceedings of the 13th USENIX Security Symposium, 2004, pp. 223–238.
25. Hardy, N. The Confused Deputy (or Why Capabilities Might Have Been Invented). *ACM SIGOPS Operating Systems Review* **1988**, *22*, 36–38. https://doi.org/10.1145/54289.871709.
26. Shannon, C.E. A Mathematical Theory of Communication. *The Bell System Technical Journal* **1948**, *27*, 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x.
27. Welford, B.P. Note on a Method for Calculating Corrected Sums of Squares and Products. *Technometrics* **1962**, *4*, 419–420. https://doi.org/10.1080/00401706.1962.10490022.
28. Page, E.S. Continuous Inspection Schemes. *Biometrika* **1954**, *41*, 100–115. https://doi.org/10.1093/biomet/41.1-2.100.
29. Kerrisk, M. *The Linux Programming Interface*; No Starch Press: San Francisco, 2010.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.